SOFTWARE ENGINEERING

NOTEBOOK

Table of Contents

CHAPTER 1

Algorithms

1 Time Complexity

1.1 Big O notation

$$f(n) = O(g(n)) \quad \text{if} \quad 0 \leq f(n) \leq cg(n) \forall n \geq n_0 \tag{1.1}$$

For all values $n >= n_0$, $f(n)$ is on or below $cg(n)$.
A constant multiple of $g(n)$ is an *asimptotic upper bound* for $f(n)$.

1.2 Omega notation

$$f(n) = \Omega(g(n)) \quad \text{if} \quad 0 \leq cg(n) \leq f(n) \forall n \geq n_0 \tag{1.2}$$

For all values $n >= n_0$, $f(n)$ is on or abow $cg(n)$.
A constant multiple of $g(n)$ is an *asimptotic lower bound* for $f(n)$.

1.3 Theta notation

$$f(n) = \Theta(g(n)) \quad \text{if} \quad 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n) \forall n \geq n_0 \tag{1.3}$$

For all values $n >= n_0$, $f(n)$ is equal to $g(n)$ within a constant factor.
$g(n)$ is an *asimptotically tight bound* for $f(n)$.

1.4 Meaning of previous notations - Insertion sort case

Big O notation
describes an *upper bound*. When we use it to *bound the worst-case running time* of an algorithm, we have a bound on the running time of the algorithm on *every input*. Thus, the $O(n^2)$ bound on worst-case running time of insertion sort also applies to its running time on every input.
The $\Theta(n^2)$ bound on the worst-case running time of insertion sort, however, does not imply a $\Theta(n^2)$ bound on the running time of insertion sort on every input. For example, when the input is already sorted, insertion sort runs in $\Theta(n)$ time. Technically, it is an abuse to say that the running time of insertion sort is $O(n^2)$, since for a given n, the actual running time varies, depending on the particular input of size n. When we say "the running time is $O(n^2)$", we mean that there is a function $f(n)$ that is $O(n^2)$ such that for any value of n, no matter what particular input of size n is chosen, the running time on that input is bounded from above by the value $f(n)$. Equivalently, we mean that the worst-case running time is $O(n^2)$.
Omega notation
describes a *lower bound*. When we say that the running time (no modifier) of an algorithm is $\Omega(g(n))$, we mean that *no matter what particular input of size n is chosen* for each value of n, the running time on that input is at least a constant times $g(n)$, for sufficiently large n. Equivalently, we are giving a lower bound on the best-case running time of an algorithm. For example, the best-case running time of insertion sort is $\Omega(n)$, which implies that the running time of insertion sort is $\Omega(n)$. The running time of insertion sort therefore belongs to both $\Omega(n)$ and $O(n^2)$, since it falls anywhere between a linear function of n and a quadratic function of n. Moreover, these bounds are asymptotically as tight as possible: for instance, the running time of insertion sort is not $\Omega(n^2)$, since there exists an input for which insertion sort runs in $\Theta(n)$ time (e.g., when the input is already sorted). It is not contradictory, however, to say that the worst-case running time of insertion sort is $\Omega(n^2)$, since there exists an input that causes the algorithm to take $\Omega(n^2)$ time.

1.5 Running times of well known algorithms

1.6 Methods for solving the recurrence

There are three methods for solving recurrences:

- the substitution method
- the recursion-tree method
- the master theorem

2 Space Complexity

CHAPTER 2

Solid Principles

SOLID is a mnemonic acronym for five design principles intended to make object-oriented designs more understandable, flexible, and maintainable.
The principles are a subset of many principles promoted by American software engineer and instructor **Robert C. Martin**, [2]- [3]- [4].
They were first introduced in his 2000 paper Design Principles and Design Patterns, while the acronym was introduced later, around 2004, by **Michael Feathers** [12].
Although the SOLID principles apply to any object-oriented design, they can also form a core philosophy for methodologies such as agile development or adaptive software development.

1 Single-Responsibility

"There should never be more than one reason for a class to change." [6]
In other words, every class should have only one responsibility. [7]

2 Open-Close

"Software entities . . . should be open for extension, but closed for modification." [8]

3 Liskov Substitution

"Functions that use pointers or references to base classes must be able to use objects of derived classes without knowing it." [9]
See also: Design by contract.

4 Interface Segregation

"Clients should not be forced to depend upon interfaces that they do not use." [10] [5]

5 Dependency Inversion

"Depend upon abstractions, [not] concretions." [11] [5]

CHAPTER 3

UML and OOP

1 UML

1.1 Class Diagram

In Unified Modeling Language, a class diagram *describes the static structure of a system* by showing the system's classes, their attributes, operations (methods) and the relationship among them.
A class diagram is the *conceptual modeling of the structure of the software application*.
In case of detailed modeling, the class diagram can be *translated into programming code*.

Visibility

+	public
–	private
#	protected
~	package

This notation is placed before methods and attributes names, to indicate their visibility.

Scope

UML defines *instance and class scopes for members*.
The latter is represented by underlined names.

- **instance members** are scoped to a specific instance
- **class members** are the one commonly recognized as `static`

Arrows

The relationship defined in UML are represented by the following logical connectors

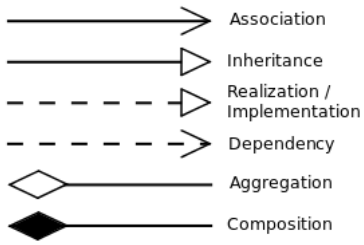


Figure 3.1: *UML logical connectors*

- **Association**
  a *relationship bewteen classes of objects* that allows one object instance to cause another *to perform an action on its behalf*.
  This relationship is structural and does not represent any behavioural trait.
  Code implementation usually requires the requsting object to invoke a method or member function using a *reference or a pointer to the controlled object*.
  Association can be *bidirectional, unidirectional, aggregation and reflexive*.
  Objects related via the association are considered to act in a role with respect to the association.
  The ends of the association can have all the characteristics of a property: *multiplicity, name, visibility and a specification* explaining whether the association is ordered and / or unique.

- **Dependency**
  a relationship showing that *an element requires other model elements* for their specification or implementation.
  The dashed lines from the dependent/client to the independent/supplier element specifies *the direction of a relationship and not of a process*.

- **Aggregation**
  a variant of *has a association relationship and more specific than the latter*. It represents a part-whole or a part-of relationship. As a type of association, an aggregation can be named and have the same adornments an association can.
  An aggregation *can occur when a class is a collection or a container of other classes*, but *the content of the container continues to exist when the container is destroyed*.

In UML notation, the *hollow diamond shape is drawn on the containing class end* of the connecting line.
An example of aggregarion is a Pond which has zero or more Ducks and Duck which has at most one Pond.
A duck can exist separately from a pond.

- **Composition**
  refers to *combining objects within compound objects, ensuring the encapsulation* of each object by *using the well-defined interface without exposing the internals*.
  Differently, data structures do not enforce encapsulation.
  Composition can be regarded as a *relationship between types*: an object of composite type *has* objects of other types
  Composition must be distinguished from subtyping, which is the process of adding detail to a general data type to create a more specific one.
  In UML notation, the *filled diamond shape is drawn on the containing class end* of the connecting line. An example of composition is a University which has one or more Departments, which belong exactly to one Univerity.
  A Department can not exist as a separate entity detached from a specific University.

Class-level relationships

- **Generalization/Inheritance**
  a relationship between two classes, in which the *subclass* is considered to be a specialization from the *super type*.
  Any instance of the subtype is also an instance of the superclass. As an example, *humans are a subclass of simian which is a subclass of mammal*.
  Generalization is also known as **inheritance** or a *is a* relationship.
  The base class is also known as parent, superclass, base class or base type.
  The subtype is also known as child, subclass, derived class, derived type, inheriting class or inheriting type.

- **Realization/Implementation**
  a relationship between two model elements, in which the *client* realizes (implements or executes) the behavior that the *supplier* specifies.

General relationship

- **Dependency**
  a weaker form of bond that indicates that one class depends on another because it uses it at some point in time. Sometimes this relationship might be implemented as member function arguments.

- **Multiplicity**
  an optional notation which can be useful to add details to the association relationship. At each end of the line, one of the following notation can be used to

indicate the range of number of objexts that partecipate in the association from the perspective of the other end.

0	No instances
0..1	No instances, or one instance
1	Exactly one instance
1..1	Exactly one instance
0..*	Zero or more instances
*	Zero or more instances
1..*	One or more instances

Class and Sequence diagram Cheatsheet

Following *a summary of the information for class and sequance UML diagrams*

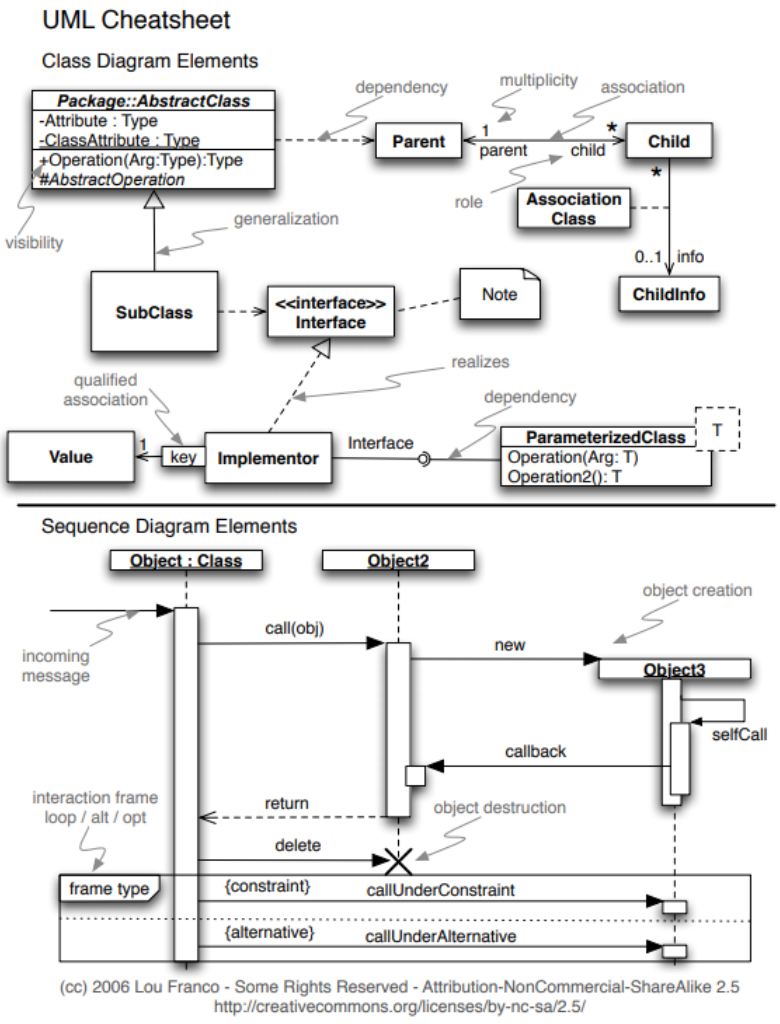


Figure 3.2: *Class and Sequence diagrams cheatsheet*

1.2 Sequence Diagram

2 Object Oriented Programming

2.1 Polymorphism

In C++ there are different kinds of polymorphism, summarized it the figure 3.3

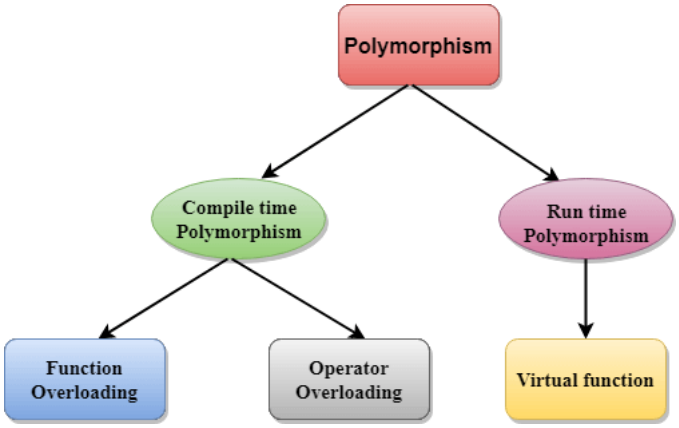


Figure 3.3: *Polimorphism in C++*

Compile time polymorphism

Overloaded functions are invoked by matching the type and number of arguments at compile time, because the compiler already has all the information needed. This is know by the name of *static binding* or *early binding*.

Run time polymorphism

2.2 Interface vs Abstract class

- Interface can contain only body-less abstract
- Abstract Class

CHAPTER 4

Design Patterns

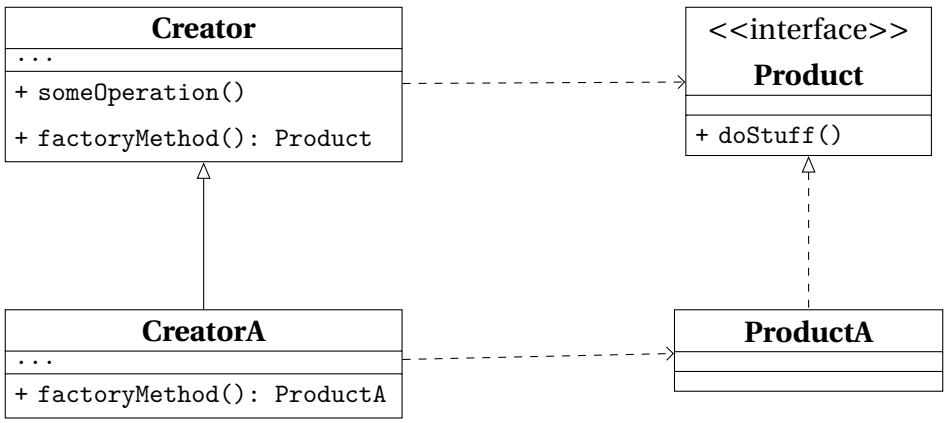
Design Patterns describe how to solve recurring design problems to design flexible and reusable object-oriented software, that is, objects that are easier to implement, change, test, and reuse.

1 Creational Patterns

1.1 Factory Method

What for	*creating objects* without having to specify the exact class of the object that will be created
How to	a *factory method* is used *instead of calling a constructor*: it can be either *specified in an interface* and implemented by child classes, or implemented in a base class and optionally overridden by derived classes
When needed	• exact types and dependencies of the objects are unknown beforehand • internal components of a library or framework need to be extended by users • system resources need to be saved by reusing existing objects.

Structure



ABCProduct	interface common to all the objects.
ConcreteProduct	different implementations of the product interface.
Creator	base class which declares the factory method: • **factory method** - abstract or returning a default product; • **returned type** - must match the interface. It can have some *core business logic* which can be splitted from the product creation.
CreatorA, **CreatorB**	concrete classes that *override the base factory method* to return the different types of concrete products.

Table 4.1: *Factory Method Class Diagram.*

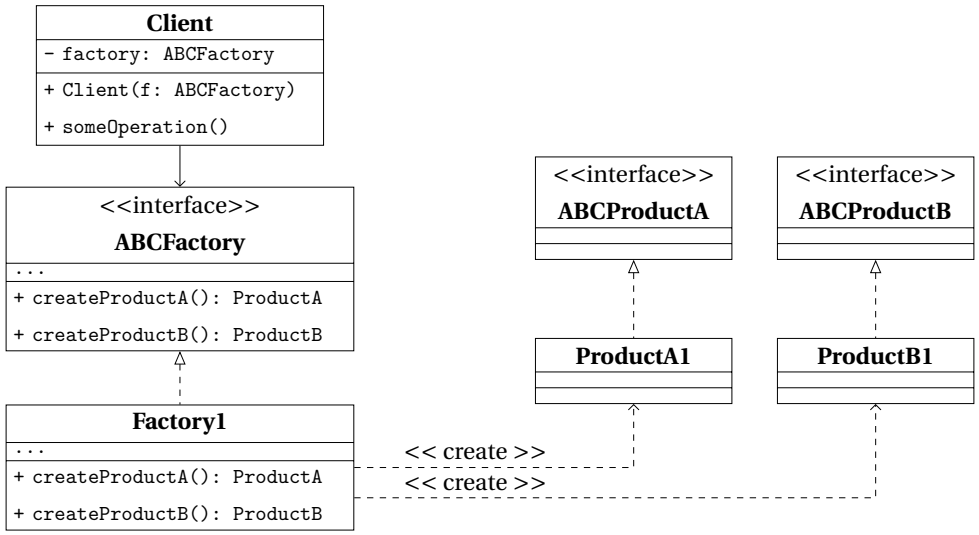
Pros and Cons

Pros	• *decoupling* of the creation of concrete objects from the core business logic of the Creator; • *Single Respobsibility Principle* • *Open/Close Principle*
Cons	code gets more complex

1.2 Abstract Factory

What for	*producing families of related items* without specifying their concrete classes
How to	declaring *explicit interfaces* for each distinct product of the product family, and making all variants of products follow those interfaces
When needed	• client needs to work with various families of related objects, without depending on the latter's concrete classes • a set of *factory methods* blurring the main responsibility of a class needs to be managed better

Structure



ABCProductA, **ABCProductB**	interfaces for a set or family of related but distinct products
ProductA1, **ProductB1**	implementations of the concrete variants of abstract products. Each abstract product must be implemented in all given variants.
ABCFactory	interface for the sets of methods needed to create each of the abstract products
Factory1	implement the creation methods of the abstract factory, each creating only the corresponding product variant
Client	can work with any concrete factory as long as it communicates with the objects through their abstract interfaces. Therefore, the signature of concrete-factories' creation-methods must match the corresponding abstract interfaces'.

Table 4.2: *Abstract Factory Class Diagram.*

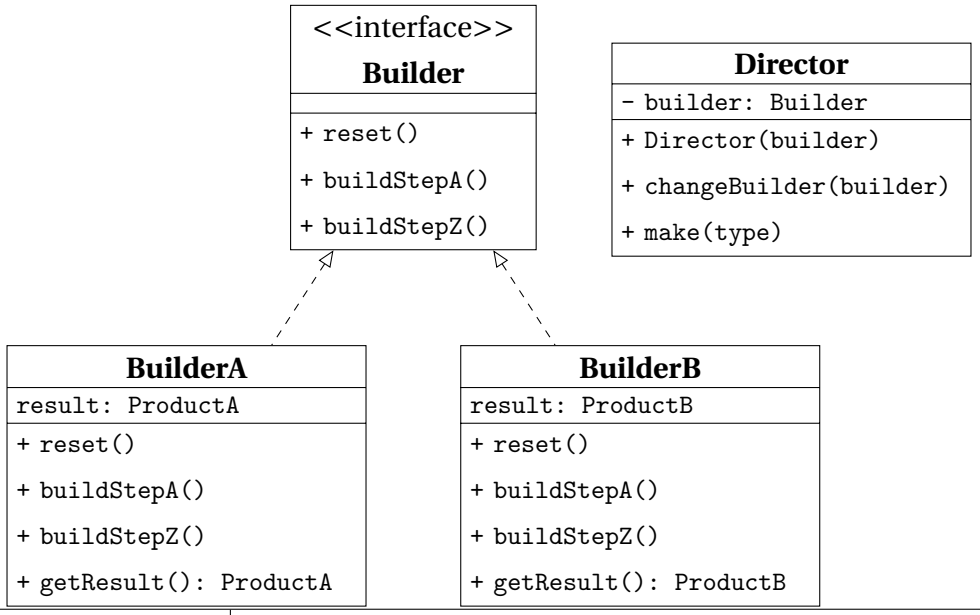
Pros and Cons

Pros	• *interchangeable concrete implementations* without changing the code that uses them • *no tight coupling* between concrete products and client code • *Single Responsibility Principle* • *Open/Close Principle*
Cons	• unnecessary complexity and work in the initial writing • systems can get more difficult to debug and maintain due to higher separation and abstraction

1.3 Builder

What for	*building complex objects step-by-step*, producing different types and representations using the same construction steps
How to	*object's construction code is moved into a Builder* class, that builds the object through steps that can be executed in series and only if needed. Several `Builders` can be implemented in case different ways of building make sense. A `Director` class can be used to manage the order of the building steps.
When needed	• ctors became telescopic because of too many optional parameters and need to be removed • code need to be able to create different representations of some products • a *composite* object construction needs to be managed step-by-step

Structure



ABCBuilder	interface that declares the production steps common to all the builders
BuilderA, **BuilderB**	implement the construction steps needed for products which do not follow the common interface
ProductA, **ProductB**	resulting objects constructed by different builders
Director	might be useful to manage the calls to the steps needed
Client	can associate a builder with a director in order to let latter be used for any further construction. An alternative way relies on passing the `Builder` object directly to the production method.

Table 4.3: *Builder Class Diagram.*

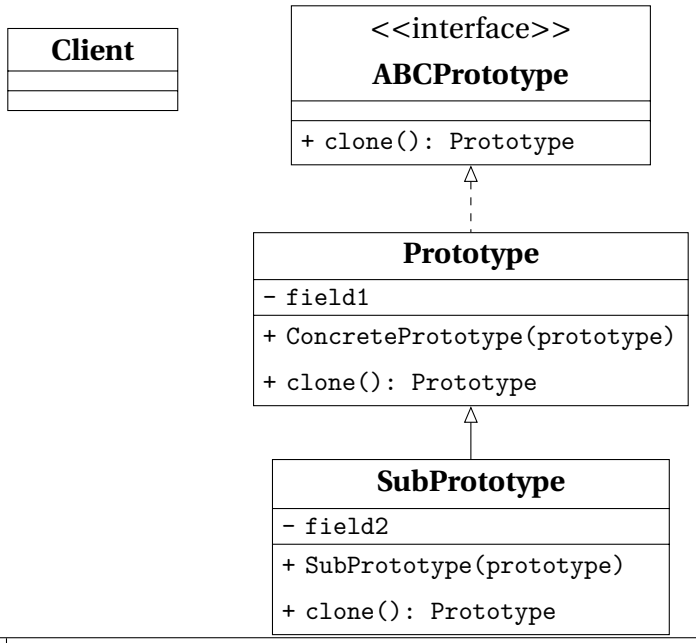
Pros and Cons

Pros	• constrution goes step-by-step and can be deferred or run recursively • construction steps can be shared in case • **Single Responsibility Principle**
Cons	multiple classes required by this pattern can increase the code complexity

1.4 Prototype

What for	*copying existing objects* without implementing code which depends on their classes
How to	a common interface is implemented for all objects that need to be cloned, and the actual cloning procedure is delegated to the objects themselves in order to have access to private and protected details as well
When needed	• code shouldn't depend on the concrete classes of the objects being copied • reducing the number of subclasses only differing in the way of initializing their respective objects.

Structure



ABCPrototype	interface that declares the cloning method
Prototype	class that implements the cloning method; it can have some logic needed to manage edge cases
SubPrototype	a class that can be handy to add subsets of prototypes
Client	can produce a copy of any object which follows the prototype interface

Table 4.4: *Prototype Class Diagram.*

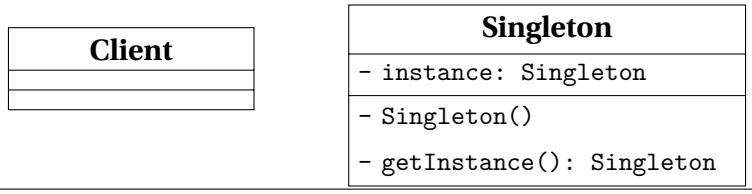
Pros and Cons

Pros	• cloning objects without coupling to their concrete classes • repeated initialization can get removed in favor of cloning pre-built prototypes • easier production of complex objects • alternative to inheritance when configuring complex objects
Cons	cloning can get more difficult when circular references are present

1.5 Singleton

What for	*ensuring that a class could have one instance only*, while providing a global access point to the instance
How to	the *ctor is made private* to prevent calls from outside the class and a *static creation method* is used in its place in order to create the object and return it for successive calls.
When needed	• a class in a program should have just a single instance available, as it can be needed for database object shared by different parts of the code • a stricter control over global variables is needed

Structure



Singleton	class declaring a `static` method that return the same instance it creates the first time it's called; this class' ctor is made `private` to hide it from external calls.

Table 4.5: *Singleton Class Diagram.*

Pros and Cons

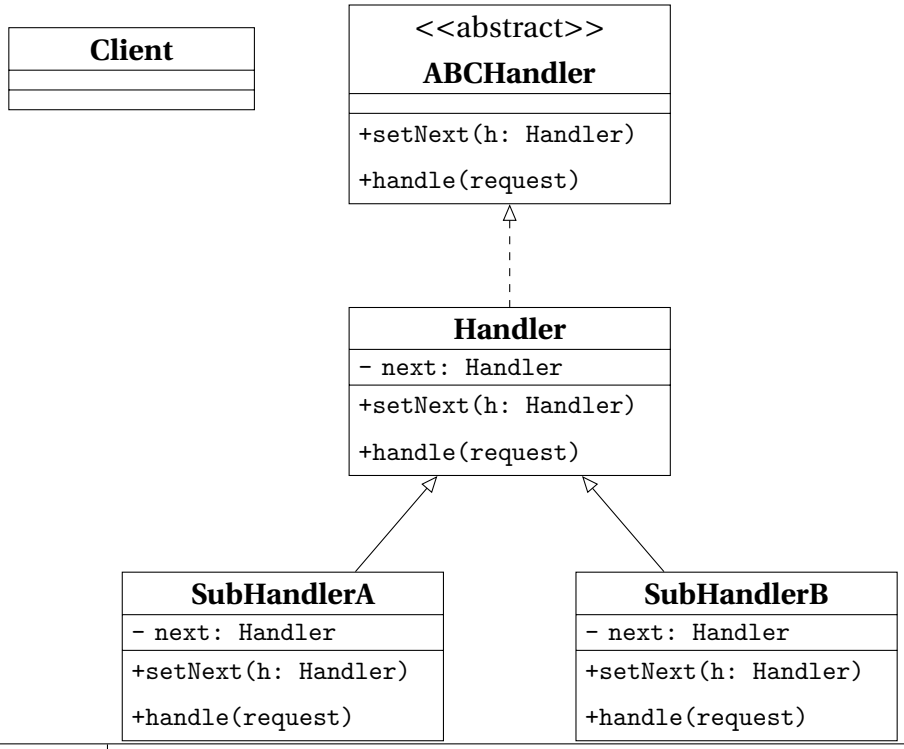
Pros	• avoiding *global namespace pollution* introducing global variables • *initialization* takes place only the first time the object is requested • *multiple but limited instances* can be created, modifing only the static method
Cons	• violation of *Single Responsibility Principle* • can mask bad design decisions, like too much overlapping between components • multithreaded programs need special care to avoid multiple instances creation • unit tests could be difficult due to the impossibility for mock objects to access the `Singleton`'s ctor.

2 Behavioral Patterns

2.1 Chain of Responsibility

What for	*pass a request along a chain of handlers*, each of which can either manage the request or pass it to the next handler in the chain
How to	each particular behavior is implemented into a stand-alone handler, the handlers are linked one to another in a chain and the request, along with the data needed, is passed along the chain
When needed	• different kind of requests are expected to be processed in different ways and the type of the request is not known beforehand • the set or the order of the handlers can change at run time

Structure



ABCHandler	common interface for all the concrete handlers, usually in terms of a first method for handling the requests and second one to set the next handler in the chain
Handler	optional base class to manage the common part of the requests
SubHandlerA, **SubHandlerB**	classes implementing the actual code to process the request or pass the request along the chain if not possible
Client	responsible of creating chains, both statically and dinamically, and of sending the request to the first or any other handler in the chain

Table 4.6: *Chain of Responsibility Class Diagram.*

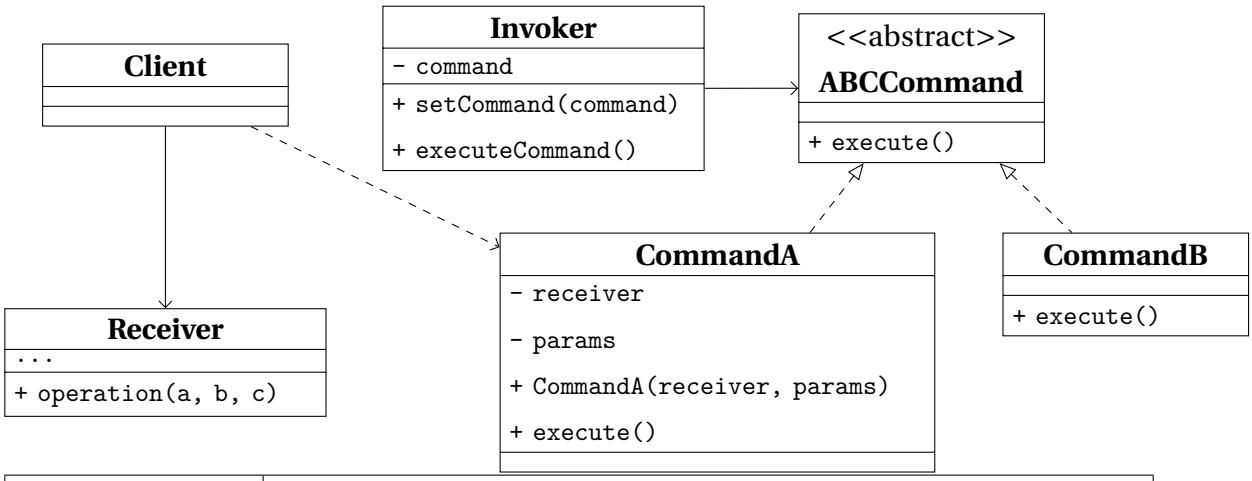
Pros and Cons

Pros	• the order of request handling can be under complete control • flexibility in deciding whether a handler should stop the request or pass it along the chain • *Single Responsibility Principle* • *Open/Close Principle*
Cons	some requests may end up unhandled

2.2 Command

What for	*turning a request into a stand-alone object* containing all information about it.
How to	applying the *Principle of Separation of Concerns*
When needed	• there is the need to parametrize objects with operations • there is the need for reversible operations

Structure



Invoker / **Sender**	class responsible for creating and sending requests, directly triggering a reference to a command object, usually provided and not created by the sender
ABCCommand	interface declaring the method to execute the command
CommandA, **CommandB**	classes implementing the concrete requests, passin the call to one of the business logic objects
Receiver	class that implements the business logic and does the actual work most of the times
Client	class responsible for creating and configuring commands, passing all the parameters needed

Table 4.7: *Command Class Diagram.*

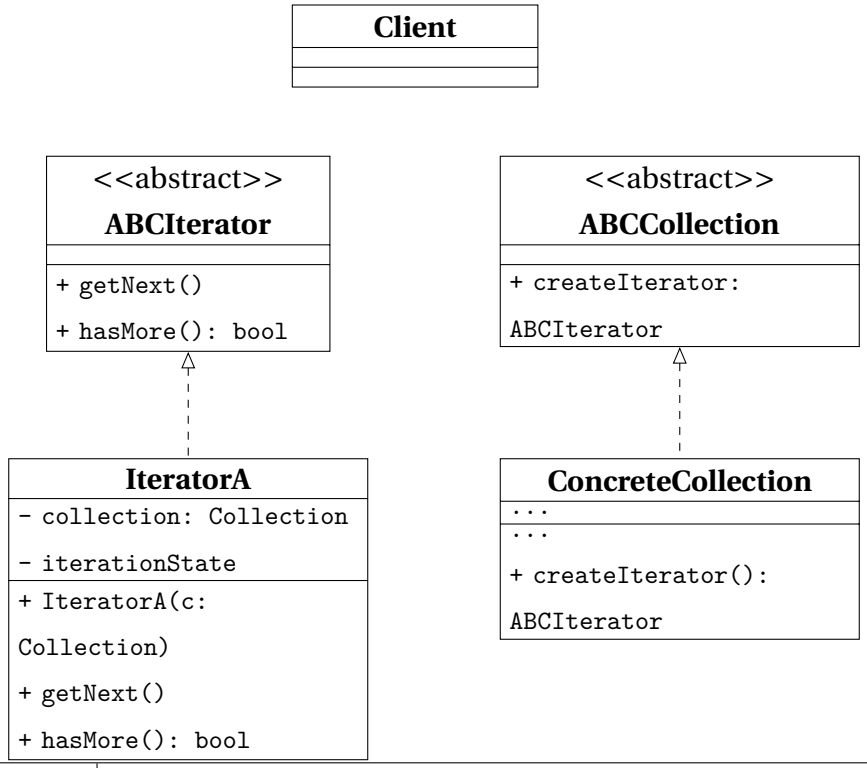
Pros and Cons

Pros	• *Single Responsibility Principle* • *Open/Close Principle* • implementing *undo*/*redo* is easier • implementing deferred execution of operation is easier • set of simple commands can be assebled in sets
Cons	programs end up having more layers

2.3 Iterator

What for	*traversing elements of a collection without exposing its underlying representation*
How to	encapsulating the details of the traversal into a object called iterator
When needed	• collection with complex data structure not to expose to users • reducing code duplication for traversals • traversing data structures, that can be unknown beforehand

Structure



ABCIterator	interface declaring the operations needed to traverse a collection, like fetching the next element, keeping track of the current position and so on
IteratorA	implements specific algorithms, independent from each others in order to be used in any number on the same collection
ABCCollection	interface declaring one or multiple methods for getting iterator compatible with the collection. The return type of these methods must match the Iterator interface.
CollectionA	returns instances of a particular concrete iterator class on client's request.
Client	can work with both the collection and the iterator using their interfaces.

Table 4.8: *Iterator Class Diagram.*

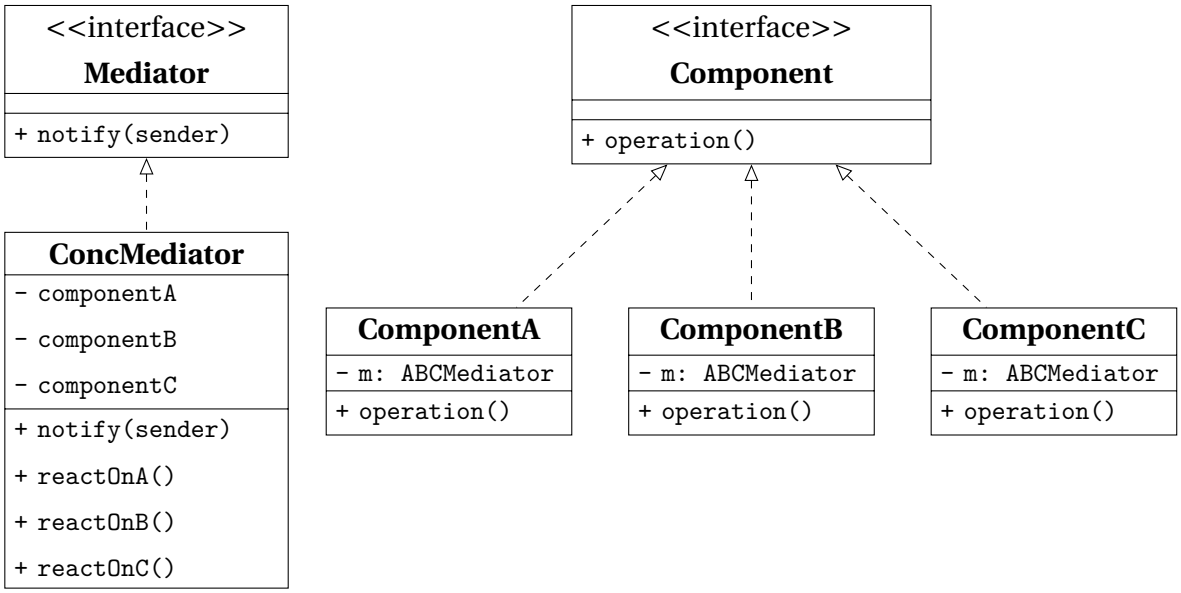
Pros and Cons

Pros	• *Single Responsibility Principle* • *Open/Close Principle* • parallel iterations are possible • iteration can be delayed to when it is needed
Cons	can be an overkill for simple collections can be less efficient than going directly through the elements of some specialized collection

2.4 Mediator

What for	*reducing chaotic and confusing dependencies* between objects and *restricting direct communications* between the objects
How to	
When needed	• changing an already exhisting class which is strongly coupled • reusing a strongly dependent component in a different program • reusing basic behaviours in completely different contexts

Structure



Component	interface for communication with other Components through its Mediator
ComponentA, . . . , **ComponentC**	implements the Colleague interface and communicates with other Components through its Mediator
Mediator	interface for communication between Component objects
ConcMediator	implements the mediator interface, coordinating communication between Component objects. It is aware of all of the Component and their purposes with regards to inter-communication

Table 4.9: *Mediator Class Diagram.*

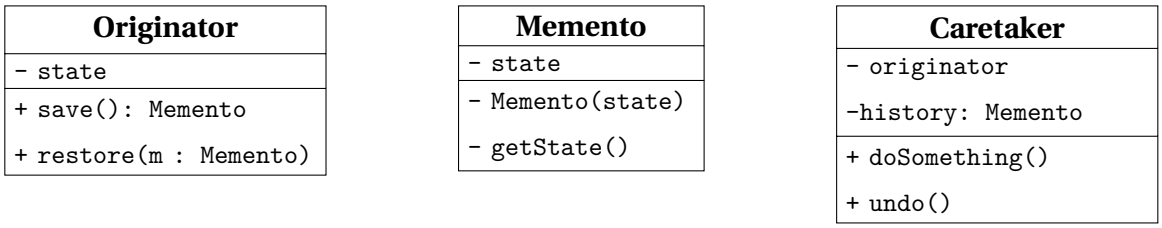
Pros and Cons

Pros	*Single Responsibility Principle* *Open/Close Principle* reducing coupling between various components reusing individual components is easier
Cons	mediator can evolve into a God object

2.5 Memento

What for	*saving/restoring states* of an object without exposing internals
How to	
When needed	• snapshots of an object's state are needed in order to restore them later • when encapsulation would be violated by directly accessing object's fields

Structure



Originator	generic class that can produce snapshots of its own state, and restore its state from one of them
Memento	class that acts like a shapshot, and could be made immutable
Caretaker	knows the *why* and *when* to capture a snapshot or restore a previous one. This class can manage an history of Originator's' states in a stack data structure.

Table 4.10: *Memento Class Diagram.*

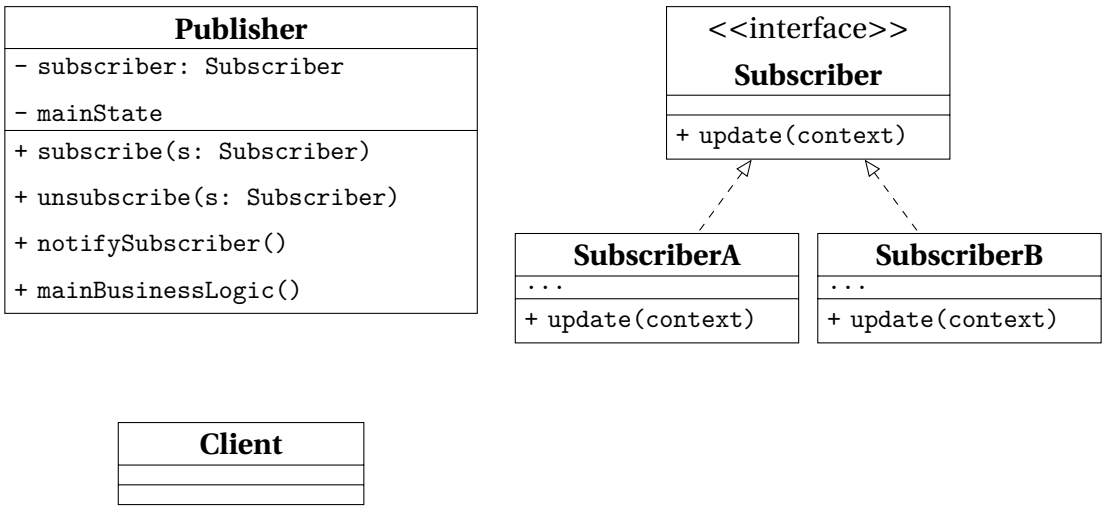
Pros and Cons

Pros	• producing snapshots of the object's state
Cons	• memory compsuntion increases • to destroy outdated mementos, the originator's lifecycle should be tracked • modern dynamic languages do not guarantee the state within the memento stays untouched

2.6 Observer

What for	implementing a *subscription mechanism*
How to	
When needed	• changes to the state of one object may require changing other objects that are unknown beforehand or change dynamically • objects must observe each others, for a limited time and in specific circumstances

Structure



Publisher	class that issues events of interests to other objects. This class contains a subscription infrastructure that lets subscribers both join and leave the subscriber list
Subscriber	interface declares the notification interface, that in most cases is a single update method with some contextual parameters.
SubscriberA, **SubscriberB**	implement the different logics in response to notifications issued by the publisher.
Client	creates both publisher and subscriber objects and manage the registrations for updates

Table 4.11: *Observer Class Diagram.*

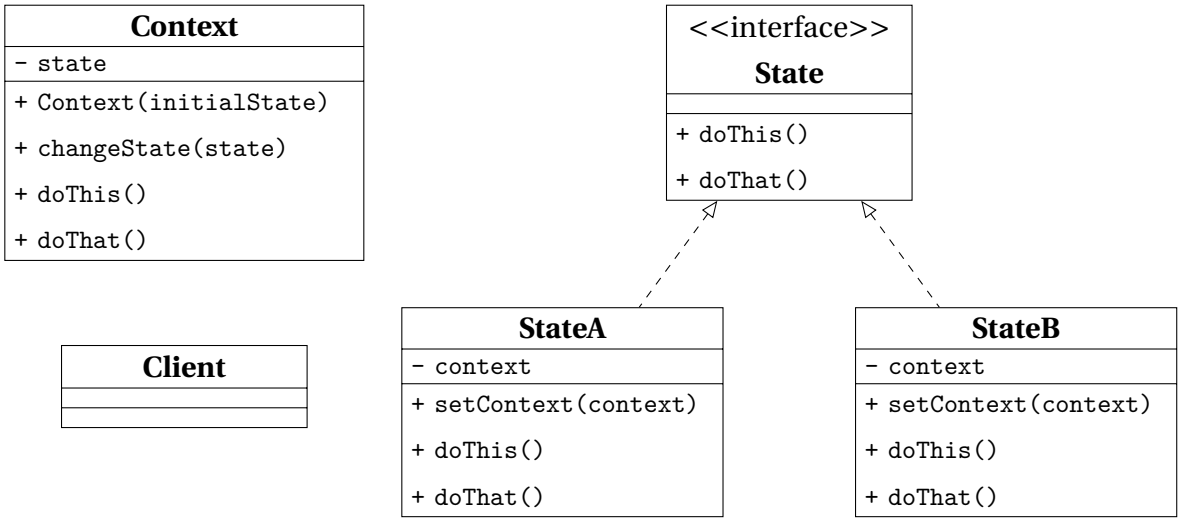
Pros and Cons

Pros	• **Open/Close Principle** • relations between objects can be established at runtime
Cons	• subscribers can be notified in random order

2.7 State

What for	implementing *objects whose behavior depends on their internal state*
How to	
When needed	• implementing an object that behaves differently depending on its current state, the number of states is considerable, and the state-specific code changes frequently • managing better a class that changes its behavior depending on some interal field actual value • managing better duplicate code across similar states and transition od a condition-besed state machine

Structure



Context	stores a reference to one of the concrete state objects, to which it delegates all the state-related work. The Context interacts with the state interface, in order to be able to interact with any Concrete State
State	interface declares the state generic methods, that should make sense for all concrete states for a matter of consistency
StateA **StateB**	implement their own specific methods, and can make use of abstract classes to encapsulate some common behavior

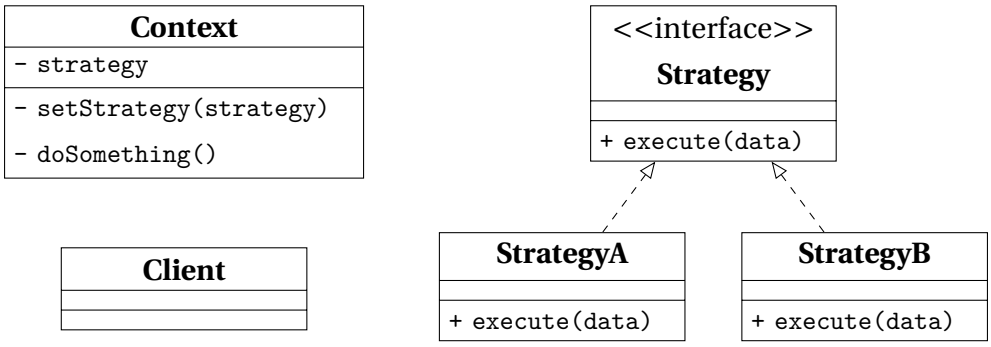
Pros and Cons

Pros	• *Single Responsibility Principle* • *Open/Close Principle* • bulky state machine code is eliminated
Cons	can be an overkill in case the state machine has a few states that change sporadically

2.8 Strategy

What for	defining a *family of algorithms, each implemented in a seprate class*, in oder to use them interchangeably
How to	
When needed	• different variants of the same algorithm must be used by an external object that can be able to switch between then • managing better lot of similar classes only differing in the way they execute some behavior • isolating business logic from the implementation details • better management of massive conditional statements that switches between variants of the same algorithm

Structure



Context	has a reference to the Strategy Interface, used to reference the Concrete Strategy in use, and methods to replace the strategy at run-time.
Strategy	interface define the part which is common to all concrete strategies and declares a method used by the Context to execute it.
StrategyA, **StrategyB**	implement a variety of algorithms needed by the context.
Client	manages the creation of the strategies and pass them to the context.

Table 4.12: *Strategy Class Diagram.*

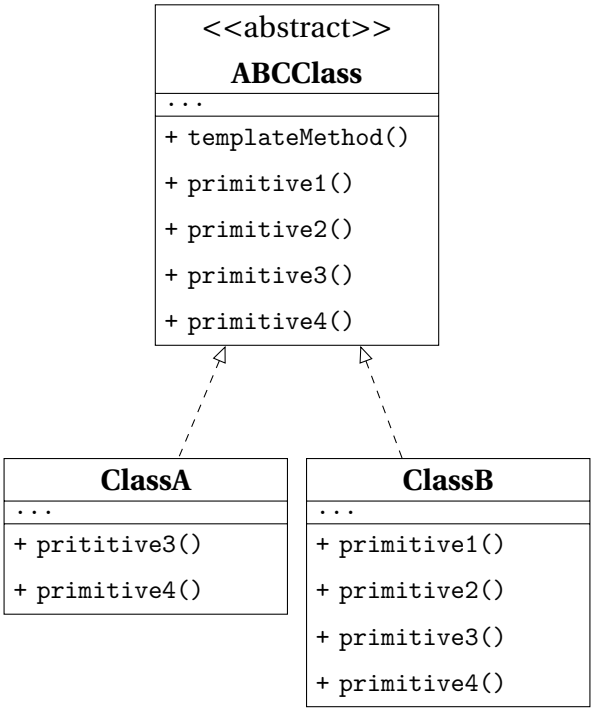
Pros and Cons

Pros	• swapping algorithms at runtime • implementation details of an algorithm is separate from the client using it • inheritance is replaced with composition • **Open/Close Principle**
Cons	• can be an overkill in case not many algorithms will be used and they will not change much at runtime • clients must know the differences between the algorithms to be able to choose • most of the modern programming languages implement different versions of an algorithm inside a set of anonymous functions and let the client use them in a similar way

2.9 Template Method

What for	*defining the skeleton of an algorithm in a class* and letting subclasses override specific steps
How to	based on *inheritance*
When needed	• client needs to extend only specific steps of an algorithm • managing better serveral classes containing almost identical algorithms

Structure



ABCClass	declares the steps of an algorithm and the `templateMethod` used to call them in a specific order. The steps may both be declared as absrtact or implement some actual routines.
ClassA, **ClassB**	override all of the steps, but not the `templateMethod` itself.

Table 4.13: *Strategy Class Diagram.*

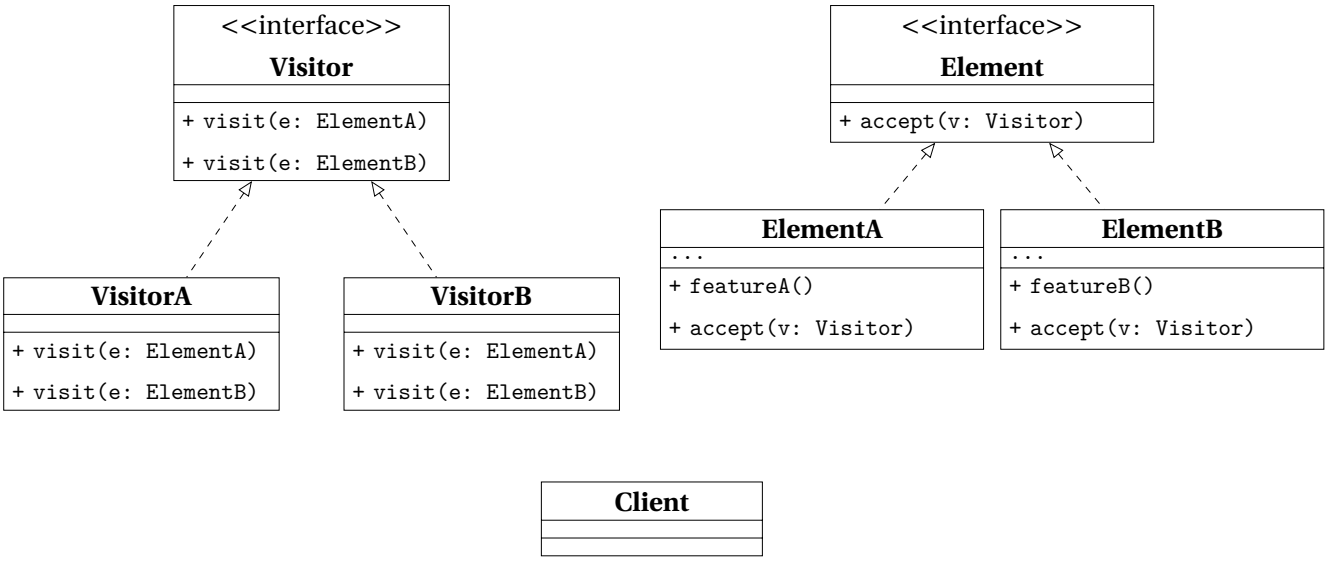
Pros and Cons

Pros	• overriding part of algorithms without affecting the rest • less code duplication
Cons	• limitations due to the skeleton of the algorithm • *Liskov Substitution Principle* • maintainance can get harder as the steps increase

2.10 Visitor

What for	*separating algorithms from the objects* they operate on
How to	
When needed	• performing an operation on all elements of a complex data structure • cleaning up the business logic extracting other behaviors • separating behaviors that make sense only for specific classes

Structure



Visitor	interface declares a set of visiting methods, each with a different Concrete Element taken as argument.
VisitorA, **VisitorB**	implement the different behaviors for each Concrete Element.
Element	interface declares a method to accept the Visitor interface as argument.
ElementA, **ElementB**	implements the acceptance method, whose purpose is the redirection the call to the proper visitor's method for the current element class. This method must be overridden in any subclass even though it was already developed in the base one.
Client	usually needs to work on a collection or a similar complex object whose type is ignored by the Client, since it works with the abstract interface.

Table 4.14: *Strategy Class Diagram.*

Pros and Cons

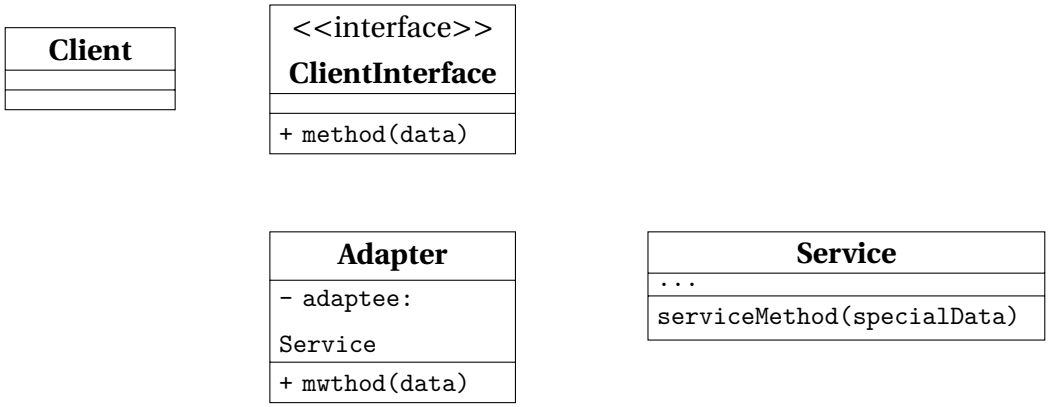
Pros	• *Open/Close Principle* • *Single Responsibility Principle* • can store information while working with different objects
Cons	• adding/removing a class requires all visitors to get updated • access grants to private data and methods of objects they are working on

3 Structural Patterns

3.1 Adapter

What for	allowing *objects with incompatible interfaces to collaborate*
How to	
When needed	• a class' interface is incompatible with the rest of the code • reusing sublcasses that lack some common functionalities that can't be added in the superclass

Structure



Client	the business logic of the program
ClientInterface	the protocol for a class to be able to collaborate with it.
Service	usually a 3rd party or legacy software with an interface incopatible with the Client.
Adapter	a class able to work with both the client and the service. It receives calls from the client and translates into calls to a wrapped service object.

Table 4.15: *Strategy Class Diagram.*

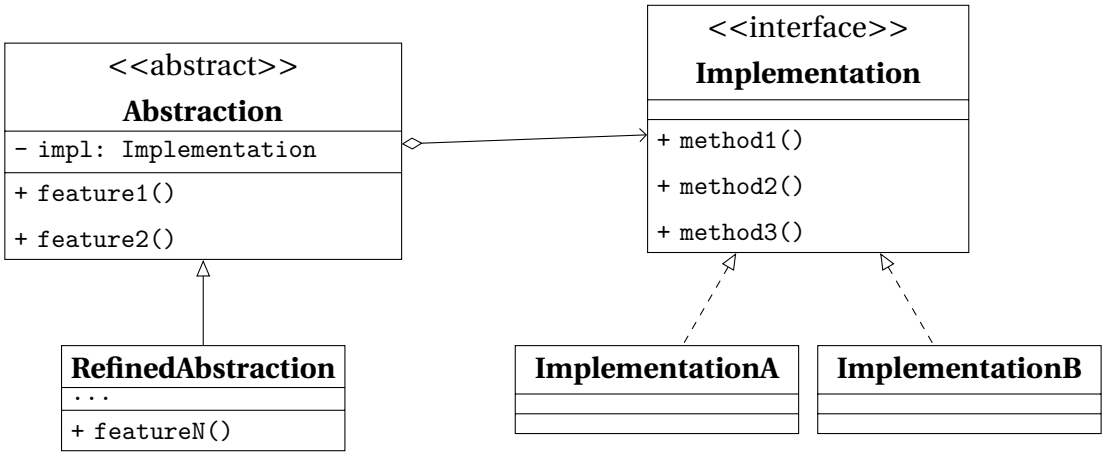
Pros and Cons

Pros	• *Single Responsibility Principle* • *Open/Close Principle*
Cons	• sometimes it's simpler to change the service class in order to match than adding a new set of interfaces

3.2 Bridge

What for	*splitting a large class or a set of related classes into two hierarchies*, an abstracion and an implementation, developed independently from each other
How to	
When needed	• dividing and organizing a monolithic class that has several variants of some functionality • extending a class in several independent dimensions • switching implementation at runtime is a requirement

Structure



Abstraction	high-level interface, and relies on the Implementation objects to perform the actual work.
RefinedAbstraction	optional and might help providing variants of control logic.
Implementation	interface shared by all the concrete implementations. Usually the abstraction declares some complex behavior that relies on primitive operations declared by the implementation.
ImplementationA, **ImplementationB**	implement specific code
Client	interacts with the abstraction, after linking it with one of the implementation objects.

Table 4.16: *Bridge Class Diagram.*

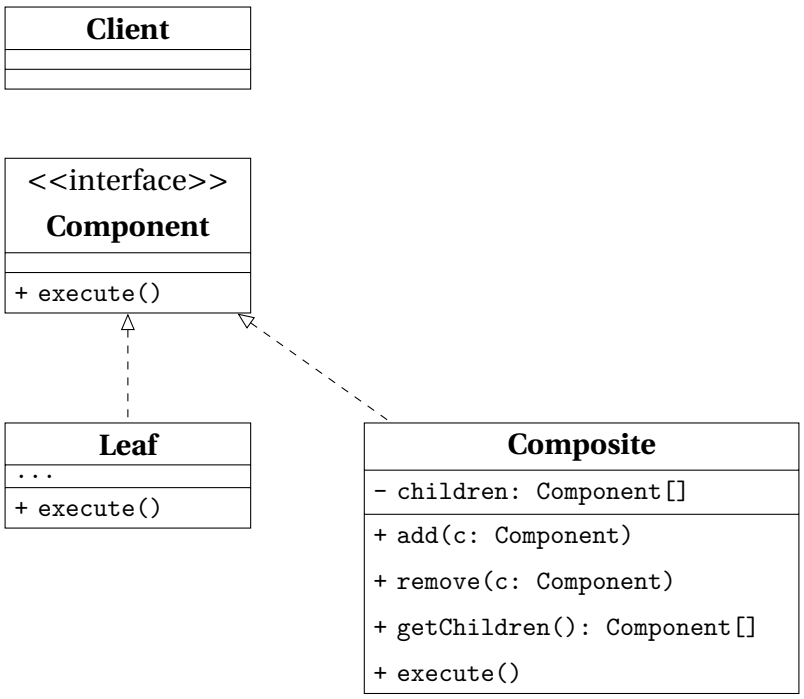
Pros and Cons

Pros	• platform-independent classes are easier to create • client isn't exposed to platform details and work with high-level abstractions • **Open/Close Principle** • **Single Responsibility Principle**
Cons	an highly cohesive class might require code getting more complicated

3.3 Composite

What for	*composing objects into tree structures* in order to work with these structures as if they were individual objects
How to	both leaf and composite objects implement the same component interface, the former working as a single entity and the latter possibly containing sub-elements, routines to add and remove them, and a routine to delegate work to them
When needed	• implementing a tree-like data structures • clients need to interact uniformly with simple and complex objects

Structure



Component	interface contains the common operations for both simple and complex elements of the tree.
Leaf	a basic element of a tree and doesn't contain any other sub-element. Usually is the part of the system which ends up doing the work.
Composite	an element that has sub-elements, which means leafs and containers as well. It interacts with sub-elements through their interfaces.
Client	can work with both simple and complex elements through the component interface.

Table 4.17: *Composite Class Diagram.*

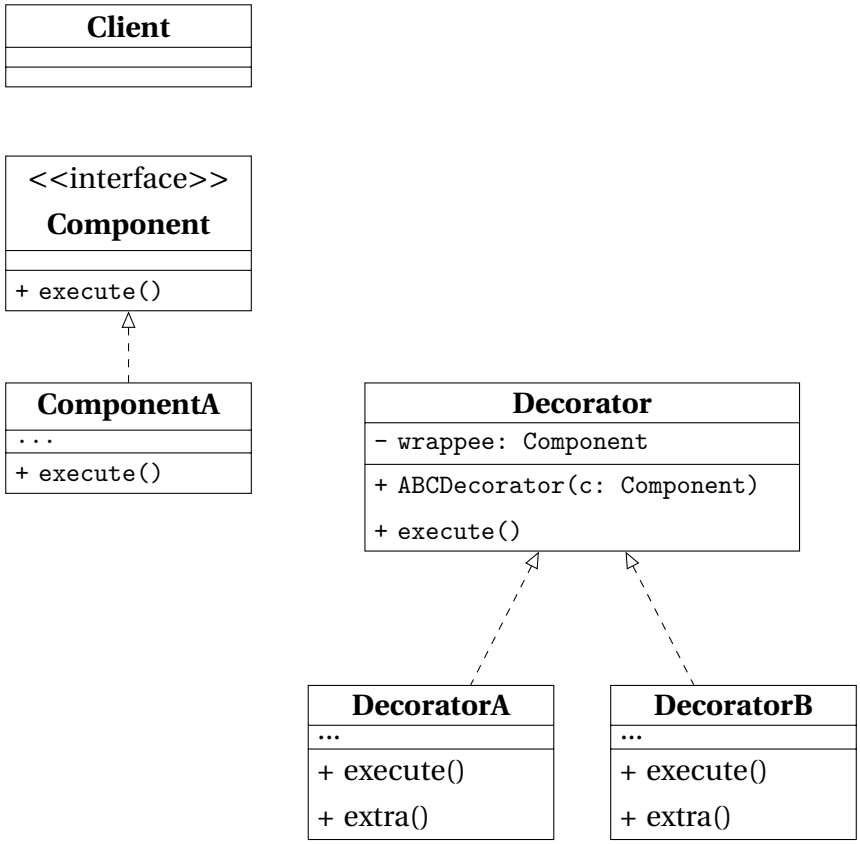
Pros and Cons

Pros	• polymorphism and recursion help work easier with complex structures • **Open/Close Principle**
Cons	a common interface can be difficult when functionalities of single elements differe too much from the ones of composite ones

3.4 Decorator

What for	*attach new behaviors to objects*, placing them in special wrappers
How to	objects are placed inside special wrapper objects that contain the behaviors
When needed	• extra behavior needs to be assigned to objects at runtime • extending object's behavior when inheritance can not be used (for example when a class is declared as `final`

Structure



Component	interface common for both wrapper and wrapped objects.
ComponentA	class being wrapped and whose basic behavior can be altered by decorators.
Decorator	has a reference of the wrapped object, whose referenced type matches the Component interface. The Base Decorator delegates all operations to the wrapped object.
DecoratorA, **DecoratorB**	define behaviors which can be added to Components dynamically. The concrete decorator overrides the base behavior and execute the variant either before of after calling the parent method.

Table 4.18: *Decorator Class Diagram.*

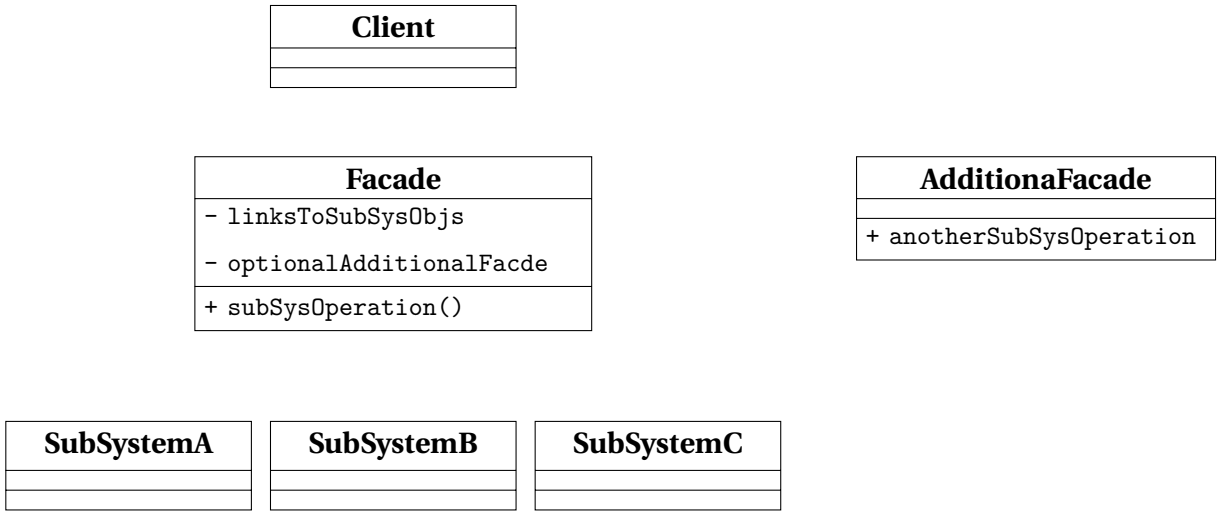
Pros and Cons

Pros	no other classes are needed to extend an object's behavior adding and removing responsibilities can be achieved at runtime ??
Cons	• removing wrappers from the stack can be difficult • implementing a decorator in a way its behavior does not depend on the order in the stack can be hard • code can get ugly when the layers increase in number

3.5 Facade

What for	providing users with a *simplified interface to any complex set of classes*
How to	a facade class provides a simple interface to a complex subsystem
When needed	• limited but straightforward interface to a complex subsystem is needed • subsystem needs to be structured into layers

Structure



Facade	implements a convenient access to a particular part of the subsystem's functionality and knows where to direct the client's request and how to operate all the system's parts.
AdditionalFacade	might help reducing the tasks managed by a single Facade. It can be used by the Client and by the other facades as well.
SubSystemA, . . . , **SubSystemC**	Subsytem class are various object which need to be studied deeply in order to manage them properly. These classes work together and are unaware of the Facade's existence.
Client	calls the subsystem classes through the Facade only

Table 4.19: *Facade Class Diagram.*

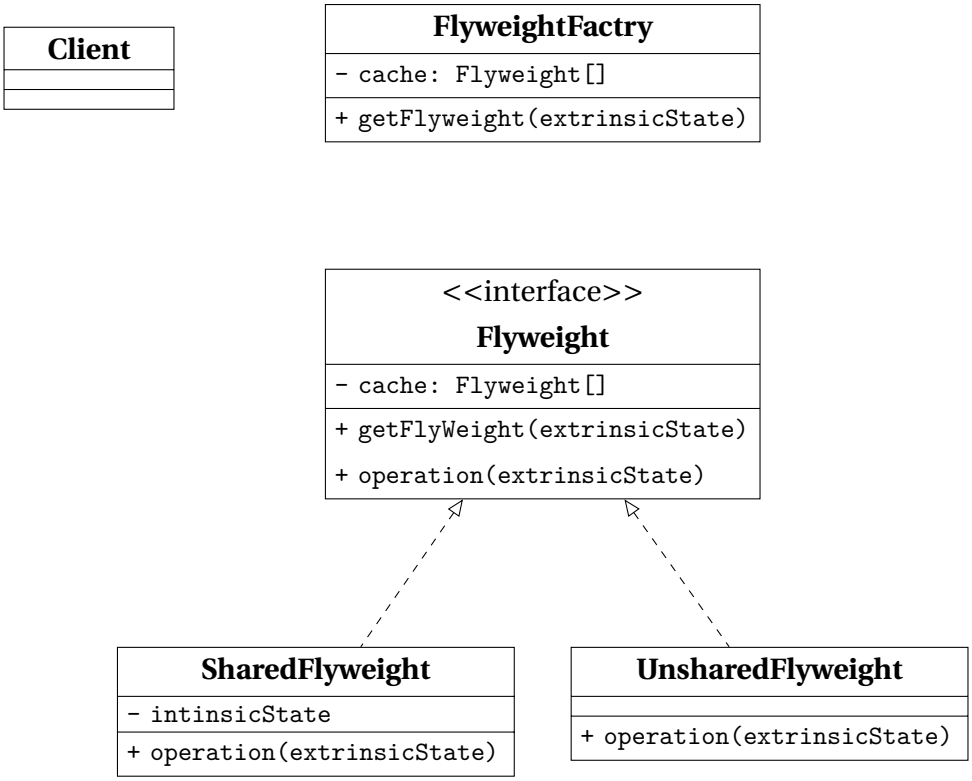
Pros and Cons

Pros	client code is isolated from the complexity of a subsystem
Cons	desire to couple a class to a facade can overwhelm the code base

3.6 Flyweight

What for	reducing RAM consumption by *sharing common parts of state between multiple objects*, instead of creating copies of same data in each object
How to	after extracting the *intrinsic state* (invariant, context-independent and shareable) an interface to the *extrinsic state* (varinat, context-dependent and unshareable) is implemented
When needed	• dealing with large numbers of objects with simple repeated elements that would use a large amount of memory if individually stored

Structure



ABCFlyweight	
SharedFlyweight	
UnsharedFlyweight	
FlyweightFactory	a class to generate and share a pool of flyweight objects
Client	

Table 4.20: *Flyweight Class Diagram.*

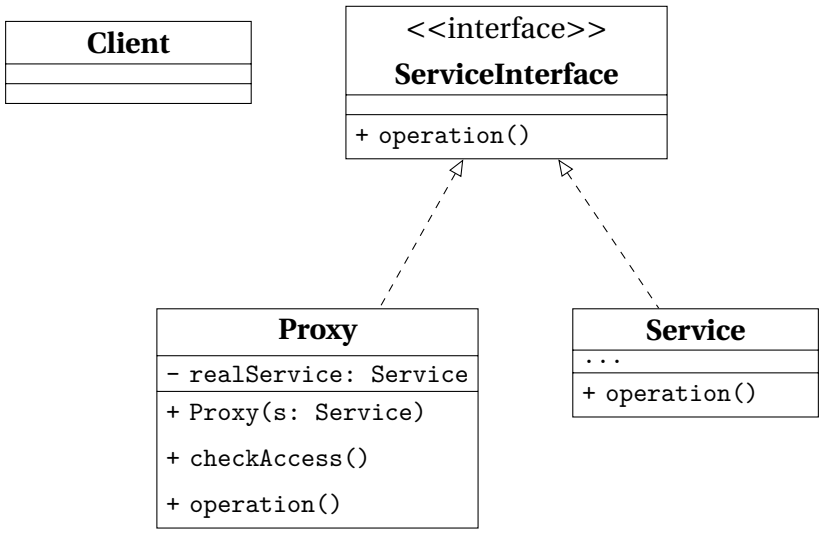
Pros and Cons

Pros	saving RAM while managing several similar objects
Cons	• CPU cycles might increase while RAM consumption decreases • code complexity and more overhead to understand the separation of states

3.7 Proxy

What for	*substitute or placeholder for another object*, *controlling access* to it giving the opportunity to make actions both before and after using the object
How to	a proxy class with the same interace of the original object is responsible of accepting request and creating real service objects
When needed	• lazy initialization • access control • local execution of a remote service • loggin the requests history through a proxy • caching requests result, expecially useful when results are large • using references to keep track of the clients using a service, in order to dismiss it when not needed

Structure



ABCServiceInterface	the interface of the service which requires a proxy, that must follow this interface to disguise itself as the service.
Service	class providing some useful business logic.
Proxy	class containing a reference to a service object, to which it passes the request after some preliminary processing and of which it manages the lifecycle.
Client	works with both the service and the proxy, if they follow the same interface.

Table 4.21: *Proxy Class Diagram.*

Pros and Cons

Pros	• control over service objects without any knoledge about their internals • lifecycle of the service object does not require management by the client • proxy can work while the service is initializing making programs faster • *Open/Close Principle* respected since new proxies can be added without changing the service or its clients
Cons	• new classes make code more complex • responses from service can be delayed

CHAPTER 5

Idioms

1 C++ Idioms

An idiom can be considered as the most natural way to express something or to solve a problem. More specifically, an idiom is a group of code fragments sharing an equivalet semantic role, which recurs frequently in one or more programming languages or libraries.

1.1 PImpl Idiom

cppreference documentation about the `PImple Idiom`.
"Pointer to implementation" or "pImpl" is a C++ programming technique that **removes implementation details of a class from its object representation** by placing them in a separate class, accessed **through an opaque pointer**.
This idiom is also know as `Compiler Firewall Idiom`.

Problematic Scenario

Whenever file class definition changes in a header, all users of that class must be recompiled. The reason is the textual inclusion on which the C++ build model is based. The user is assumed to know two features which could be affected by private members: **size and layout** and **functions**.

Solution

```
// Pimpl idiom - basic idea
class widget {
    // :::
private:
    struct impl;         // things to be hidden go here
    impl* pimpl_;        // opaque pointer to forward-declared class
};
```

1.2 Fast PImpl Idiom

1.3 Handle/Object Idiom

1.4 Copy and Swap Idiom

This idiom aims at creating a strong exception-safe implementation of the overloaded assignment operator.
This idiom makes use of the **RAII** idiom in order to acquire the new resource before it forfeits its current resource. If the acquisition is successful

1.5 RAII - Resource Acquisition is Initialization

1.6 DBDII - Dynamic Binding During Initialization Idiom

calling-virtuals-from-ctor-idiom

1.7 Named Constructor Idiom

[22]

1.8 Construct On First Use Idiom

[23]

1.9 Nifty Counter Idiom

[24]

1.10 Named Parameter Idiom

[26]

1.11 Virtual Friend Function Idiom

[27]

CHAPTER 6

Containers

The Containers library is a generic collection of class templates and algorithms that allow programmers to easily implement common data structures like queues, lists and stacks.
The container classes are:

- sequence containers
- associative containers, which can be searched in $O(\log n)$.
- unordered associative containers, which can be searched in $O(1)$ on average and in $O(n)$ in the worst one.

1 Links to the CPP Reference documentation

1.1 Sequence Containers

array | vector | deque | forward_list | list

1.2 Associative Containers

set | map | multiset | multimap

1.3 Unordered Associative Containers

unordered_set | unordered_map | unordered_multiset | unordered_multimap

1.4 Container Adaptors

stack | queue | priority_queue | flat_set | flat_map | flat_multiset | flat_multimap

2 Containers' Access Time

Associative Container	Sorted	Value	Multiple Identical Keys	Access Time
`std::set`	yes	no	no	logarithmic
`std::unordered_set`	no	no	no	constant
`std::map`	yes	yes	no	logarithmic
`std::unordered_map`	no	yes	no	constant
`std::multiset`	yes	no	yes	logarithmic
`std::unordered_multiset`	no	no	yes	constant
`std::multimap`	yes	yes	yes	logarithmic
`std::unordered_multimap`	no	yes	yes	constant

3 Invalidation of Iterators and References

Category	Container	Valid after **insertion**		Valid after **erasure**		Conditionality
		iterators	reference	iterators	reference	
Sequence Containers	array	N/A		N/A		
	vector	No		N/A		Insertion changed capacity
		Yes		Yes		Before modified element(s) (for insertion only if capacity didn't change)
		No		No		At or after modified element(s)
	deque	No	Yes	Yes, but erased one(s)		Modified first or last element
			No	No		Modified middle only
	list	Yes		Yes, but erased one(s)		
	forward_list	Yes		Yes, but erased one(s)		
Associative Containers	set multiset map multimap	Yes		Yes, but erased one(s)		
Unordered associative Containers	unordered_set unordered_multiset unordered_map unordered_multimap	No	Yes	N/A		Insertion caused rehash
		Yes		Yes, but erased one(s)		No rehash

4 C++23 New Container Adaptors

4.1 Containers

With C++23, the following four Container Adaptors were introduced:

C++23	`flat_map`, `flat_multimap`, `flat_set`, `flat_multiset`

CHAPTER 7

Memory Management

1 Memory Management

Allocation means to assign, allot, distribute or "set apart for a particular purpose". Programs manage their memory paritioning or dividing it into different units performing specific tasks.
In C++, there are two ways to create objects and this aspect reflects on the existence of two different units used to allocate the memory, the **stack** and **heap**, which manage the program's used/unused memory in two different ways.

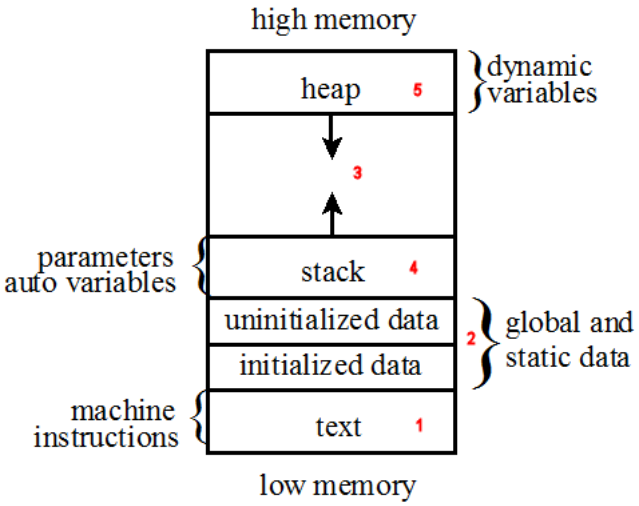


Figure 7.1: *Functional memory organization of a running program*

Text

This area contains machine instructions (i.e. the executable code). **Global** and **static** variables are typically allocated at program startup. **Stack** contains all the automatic variables. **Heap** is meant to allocate the memory for the dynamic variables through the `new` and `delete` operators. The space in between is allocated by the operating system but not used, and is meant to let both the heap and stack grow.

Stack

This is a simple LIFO (last in, first out) data structure which must support at least the **push** and **pop** operations. Even though a stack only allows accesses at the top, it is easy to implement and fast to execute push and pop operations.

Heap

This is a more flexible data structure, which allows programs to allocate or deallocate memory anywhere and in any convenient order. This might cause hole in the heap, i.e. unallocated memory surrounded by allocated memory.

The only restriction for the heap is that memory can be allocated in a contiguous block of memory large enough to satisfy the request with a single chunk of memory. This restriction has two effects: it requires the operating system to scan the heap until a contiguous block is found, and to coalesce adjacent blocks of memory returned by the program.

Side effect is that the operation on the heap are slower than the operations on the stack.

1.1 Memory Allocation and Deallocation in C

`malloc`	
`free`	

1.2 Memory Allocation and Deallocation in C++

new expression	
delete expression	

CHAPTER 8

Smart Pointers

1 Smart Pointers in C++

Cppreference documentation about **smart pointers**.
Smart pointers enable automatic, exception-safe, object lifetime management.
Generally speaking, a **smart pointer** is a data type whose value can be used like a pointer, but which provides the additional feature of memory management which frees the memory when the smart pointer is no longer used. A smart pointer should therefore be used any time the ownership of memory needs to be tracked.

1.1 `std::unique_ptr`

cppreference documentation about `std::unique_ptr`.
`std::unique_ptr` is a smart pointer that **owns and manages** another object through a pointer and **disposes of** that object when the `std::unique_ptr` goes out of scope.
The object is disposed of, using the associated deleter when either of the following happens:

- the managing `std::unique_ptr` object is destroyed

- the managing `std::unique_ptr` object is assigned another pointer via `operator=` or `reset()`.

Notes

Only **non-const** `std::unique_ptr` can **transfer the ownership** of the managed object to another `std::unique_ptr`.
If an object's lifetime is managed by a const `std::unique_ptr`, it is limited to the scope in which the pointer was created.
`std::unique_ptr` is commonly used to manage the lifetime of objects, including:

- providing **exception safety** to classes and functions that handle objects with dynamic lifetime, by **guaranteeing deletion** on both normal exit and exit through exception
- **passing ownership** of uniquely-owned objects with dynamic lifetime into functions
- **acquiring ownership** of uniquely-owned objects with dynamic lifetime from functions
- as the element type in move-aware containers, such as `std::vector`, which hold pointers to dynamically-allocated objects (e.g. if **polymorphic behavior** is desired).

`std::unique_ptr` may be constructed for an **incomplete type** T, such as to facilitate the use as a handle in the pImpl idiom.

Member types

pointer	Must satisfy NullablePtr
element_type	T, the type of the object managed by this `std::unique_ptr`
deleter_type	Deleter, the function object or lvalue reference to function or to function object, to be called from the destructor

Member functions

constructor	constructs a new `unique_ptr`
destructor	destructs the managed object if such is present
operator=	assigns the `unique_ptr`

Modifiers

release	returns a pointer to the managed object
reset	returns the deleter that is used for destruction of the managed object
swap	checks if there is an associated managed object

Observers

get	returns a pointer to the managed object
get_deleter	returns the deleter that is used for destruction of the managed object
operator bool	checks if there is an associated managed object

Single-object versioin, `unique_ptr`

operator* operator->	dereferences pointer to the managed object

1.2 `std::shared_ptr`

Cppreference documentation about `std::shared_ptr`.
This is a smart pointer that retains shared ownership of an object through a pointer.
Several `shared_ptr` objects may own the same object.
The object is destroyed and its memory deallocated when either of the following happens:

- the last remaining `shared_ptr` owning the object is destroyed;
- the last remaining `shared_ptr` owning the object is assigned another pointer via `operator=` or `reset()`.

A `std::shared_ptr` can share ownership of an object while storing a pointer to another object.
A `std::shared_ptr` may also own no objects, in which case it is called empty (an empty `std::shared_ptr` may have a non-null stored pointer if the aliasing constructor was used to create it).

Member functions

constructor	constructs a new `unique_ptr`
destructor	destructs the managed object if such is present
operator=	assigns the `unique_ptr`

Modifiers

reset	replaces the managed object
swap	swaps the managed objects

Observers

get	returns the stored pointer
operator* operator->	dereferences the stored pointer
operator[]	provides indexed access to the stored array
use_count	returns the number of `shared_ptr` objects referring to the same managed object
unique	checks if the managed object is managed only by the current `shared_ptr` instance
operator bool	checks if the stored pointer is not null
owner_before	provides owner-based ordering of shared pointers

1.3 `std::weak_ptr`

cppreference documentation about `std::weak_ptr`.
`std::weak_ptr` is a smart pointer that holds a non-owning ("weak") reference to an object that is managed by `std::shared_ptr`. It must be converted to `std::shared_ptr` in order to access the referenced object.
`std::weak_ptr` models temporary ownership

Member functions

constructor	creates a new `weak_ptr`
destructor	destroys a `weak_ptr`
operator=	assigns a `weak_ptr`

Modifiers

reset	releases the ownership of the managed object
swap	swaps the managed objects

Observers

use_count	returns the number of `shared_ptr` objects that manage the object
expired	checks whether the referenced object was already deleted
lock	creates a `shared_ptr` that manages the referenced object
owner_before	provides owner-based ordering of weak pointers

1.4 `std::auto_ptr`

cppreference documentation about `std::auto_ptr`.
It was deprecated in C++11 and removed in C++17.

CHAPTER 9

C++

1 C++: General Knowledge and Good Practices

1.1 Rule of three/five/zero

References in bibliography: [33], [34], [35], [36].

Rule of three

If a class requires a *user-defined destructor*, a user-defined *copy constructor*, or a user-defined *copy assignment operator*, it almost certainly requires all three.
Because C++ copies and copy-assigns objects of user-defined types in various situations (passing/returning by value, manipulating a container, etc), these special member functions will be called, if accessible, and if they are not user-defined, they are implicitly-defined by the compiler.
The implicitly-defined special member functions are typically incorrect if the class manages a resource whose handle is an object of non-class type (raw pointer, POSIX file descriptor, etc), whose destructor does nothing and copy constructor/assignment operator performs a "shallow copy" (copy the value of the handle, without duplicating the

underlying resource).
Classes that manage non-copyable resources through copyable handles may have to declare copy assignment and copy constructor private and not provide their definitions or define them as deleted. This is another application of the rule of three: deleting one and leaving the other to be implicitly-defined will most likely result in errors.

Rule of five

Because the presence of a user-defined destructor, copy-constructor, or copy-assignment operator prevents the implicit definition of the move constructor and the move assignment operator, *any class for which move semantics are desirable*, has to declare all five special member functions.
Unlike Rule of Three, failing to provide move constructor and move assignment is usually not an error, but a missed optimization opportunity.

Rule of zero

Classes that have custom destructors, copy/move constructors or copy/move assignment operators should deal exclusively with ownership (which follows from the Single Responsibility Principle). Other classes should not have custom destructors, copy/move constructors or copy/move assignment operators. (see [33])
This rule also appears in the C++ Core Guidelines as C.20: If you can avoid defining default operations, do.
When a *base class is intended for polymorphic use*, its destructor may have to be declared public and virtual. This blocks implicit moves (and deprecates implicit copies), and so the special member functions have to be declared as defaulted (see [34])
However, this makes the class prone to slicing, which is why polymorphic classes often define copy as deleted (see [35]), which leads to the following generic wording for the Rule of Five: If you define or =delete any default operation, define or =delete them all (see [36])

1.2 The Most Vexing Parse

Examples

How to fix it

1.3 Const Correctness

ISOCpp FAQ about `const` correctness.
Const correctness consists in using the keyword `const` to prevent `const` objects from being mutated.

const and pointer combinations

Reading it right-to-left

`const X* p`	p is a pointer to an X that is constant.
`X const* p`	equivalent to `const X* p`.
`X* const p`	p is a `const` pointer to an X that is not `const`.
`const X* const p`	p is a `const` pointer to an X that is `const`.

The second combination is equivalent to the first one: the `const-on-the-right style` requires to write the `const` specifier on the right of the object it makes `const`. This style is also called `consistent const` since it is consistent with the way of defining a const member function. Moreover, it is also more consistent with pointer declarations.

const and reference combinations

Reading it right-to-left The equivalence between the second and the first combination

`const X& r`	r aliases an X object, which can not be changed via r
`X const& r`	equivalent to `const X& r`
`X& const r`	nonsense functionally equivalent to `X& r`
`const X& const r`	nonsense functionally equivalent to `const X& r`

is due to the same reason valid for the pointers.
The last two combinations are nonsense and should be avoided. They are redundant as a reference is always constant, in the sense that you can never reseat a reference to make it refer to a different object.

Logical and Physical State

When reasoning in terms of const-correctness, it's important to distinguish the logical and the physical state of an object.
On the inside, an object has a **physical state**, also called *concrete* or *bitwise* state. This state is the one visible to the programmer who implements the software. From the outside, the client has an access to the object restricted to public member functions and friend functions. What is visible is the **logical** or *abstract* or *meaningwise state*.
A lookup functionality for a collection-object is a good example to show how there could be bits in the object's physical state that haev no corresponding elements in the object's logical state. The lookup functionality might improve the performance using a cache to store the already looked-up items. The implementation of the cache is part of the object's physical state but the implementation is internal and the details will probably not be exposed to the users, i.e. it is not part of the object's logical state.

`const`ness of `public` member functions

The constness of a public method must make sense to the object's users, and those users can see only the object's logical state. Therefore, in this context the `const`ness to take care of is the logical one.

- If a method changes any part of the object's logical state, it logically is a mutator; it should not be const even if (as actually happens!) the method doesn't change any physical bits of the object's concrete state.
- Conversely, a method is logically an inspector and should be const if it never changes any part of the object's logical state, even if (as actually happens!) the method changes physical bits of the object's concrete state.

return-by-reference and `const` member function

A `const` member function must not change nor allow its caller to change the logical state of the `this` object it's been called on.
If an inspector method needs to return a data member of a `this` object, it should return by `const reference` or by copy. For example

```
class Person
{
    public:
        const std::string& get_name() const;  // good one
        std::string& get_lastname() const;    // wrong one
        int get_age();                        // good one
}
```

Compilers won't always catch such a mistake, therefore it is responsibility of the programmer.

`const` overloading

`const` overloading is a way to achieve `const` correctness when there are an inspector and mutator method with the same name. One of the most common case is due to the subscript operators, which usually come in pair as in the following

```
class PersonList
{
    public:
        // Subscript operators often come in pairs
        const Person& operator[] (unsigned index) const;
        Person&       operator[] (unsigned index);
        // ...
};
```

Of course `const`-overloading can be used for other things than subscript operator.
Following some links to the `isocpp FAQs`

- Subscript operator for Matric class

- Matrix class made simpler
- Matrix using templates
- Matrix using templates and `vectors`
- Class Template's syntax and semantics

`mutable` keyword

`mutable` keyword helps achieving the `const` correctness, when the code needs to make changes to a `const` object's physical state. An example could be again the cached previous lookup in 1.3. In this case, the cache would be marked as `mutable` and the compiler would let the programmer change it even in case of a `const` object because it would know that a change of that attribute would reflect in a change in the physical state, leaving the logical state as it was before.
Using `const` is better than removing the constness with a `const_cast`

Error converting `F** → const F**`

C++ allows the conversion `F** → F const*` but gives an error if a program tries to implicitly convert `F** → const F**`.
The reason why this conversion is not legal in C++ is that it would let silently and accidentally modify a `const Foo` object without a cast.

```
class Foo
{
    public:
    void modify();  // make some modification to the this object
};

int main()
{
    const Foo x;
    Foo* p;
    const Foo** q = &p;  // q now points to p;
                         // this is (fortunately!) an error
    *q = &x;             // p now points to x
    p->modify();         // Ouch: modifies a const Foo!!
    // ...
}
```

In case the third line of the main were legal, `q` would be pointing at `p` and the next line it would change `p` to point at `x`. This would maje the `const` qualifier disappear. The last line would then use `p` ability to modify its referent and the program would end up modifying a `const Foo`.
In case of necessity, the solution is to change `const Foo**` to `const Foo* const*`.

1.4 Classes and Inheritance

Classes and Objects

ISOCpp FAQ about **Classes and Objects**

Constructors

ISOCpp FAQ about **Constructors**

Destructors

ISOCpp FAQ about **Destructors**

Assignment Operator

ISOCpp FAQ about **Assignment Operator**

Operator Overloading

ISOCpp FAQ about **Operator Overloading**

Friends

ISOCpp FAQ about **Friends**
Cppreference documentation about **Friends**
A `friend` can be a `function`, a `class`, and both their `template` version.
The friend declaration appears in a class body and grants a function or another class access to `private` and `protected` members of the class where the friend declaration appears.
`friend` declaration helps enforcing encapsulation. Indeed, granting access to `private` and `protected` members removes the burden of creating public `getters` and `setters`, since in most of the cases these methods destroy encapsulation, exposing private and protected details which make no sense outside of the class.
The disadvantage of `friend` functions is that they require extra code when dynamic binding is needed. Indeed, the `friend` function should call a hidden (usually `protected`) `virtual` member function, as explained in details in 1.11.
Friendship *is not inherited, transitive or reciprocal*:

- a derived class of a friend class is not a friend class;
- a friend of a friend is not necessarily a friend;
- if `class A` declares `class B` its friend, `B` objects have access to `A` objects' private data, but the opposite is not true.

Usually, it is better to rely on member function if possible and `friend` ones when it is really needed.

Inheritance - Basics

ISOCpp FAQ about **Inheritance - Basics**

`static` initialization

Inheritance - Virtual member function

ISOCpp FAQ about Inheritance - `virtual` **functions**.
A virtual member function is the way of C++ to allow derived classes to replace the implementation provided by the base class. The compiler makes sure the correct derived class' implementation is called, even when the object is accessed through a pointer to the base class.
The reason why the member function are not `virtual` by default is that many classes are not designed to be used as a base class. Moreover, the `virtual` function call mechanism needs requires an overhead of memory, typically one word per object.

Dynamic Binding and Static Typing

A pointer to an object may actually point to a class that is derived from the class of the pointer. Thus there are two different types to consider: the (static) type of the pointer and the (dynamic) type of the actual pointed object.
Static typing means that the legality of a member function invocation is checked as early as possible, i.e. at compile time by the compiler itself. In this kind of check, the compiler uses the *static type* of the pointer to determine whether the member function invocation is correct, meaning that the type pointed by the pointer can handle the member function.
Dynamic binding is the `virtual` function call mechanism, which checks the correctness of the function call at the last possible moment, i.e. at run time and based on the dynamic type of the object pointed to.

Member Function call: `virtual` vs. non-`virtual`

The practical difference is that non-`virtual` member functinos are resolved statically, i.e. the compiler checks the *type of the pointer or reference to the object* and selects the correct member function statically at compile time.
On the other side, a `virtual` member function call is resolved dynamically.

`virtual` Member Function Call - Overhead Estimation

As explained, a non-`virtual` member function is resolved statically based on the type of the pointer or reference to the object.
In contrast, **dynamic binding** select the correct `virtual` member function at run time following a compiler-based variant of the following technique: for each virtual function, the compiler create in each object an hidden *v-pointer* (virtual pointer) pointing to a global table called *v-table* (virtual table). One single *v-table* is created for each class

which contains at least a `virtual` function.
To resolve a call to a *virtual* function, the run-time procedure follows the object's *v-pointer* to the class's *v-table*, and then follows the appropriate slot in the *v-table* to the method code.
The **space-cost** overhead consists of an extra pointer per object needing to be dynamically binded and another extra pointer per `virtual` method. The **time-space** overhead compared to a normal function call consits of the two extra fetches to get the value of the *v-pointer* and a second one to get the address of the method.
Suppose to have a `Base` class with 5 `virtual` functions

```
// Original C++ source code
class Base
{
    public:
        virtual return_type virt0( /*...arbitrary params...*/ );
        virtual return_type virt1( /*...arbitrary params...*/ );
        virtual return_type virt2( /*...arbitrary params...*/ );
        virtual return_type virt3( /*...arbitrary params...*/ );
        virtual return_type virt4( /*...arbitrary params...*/ );
        // ...
};
```

There are three steps the compiler goes through

First step: building a static table
Compiler build a static table containing 5 function-pointers, most likely while compiling the `.cpp` that define the `Base`'s first non-`inline` `virtual` function.
Let's call this table `Base::__vtable`. If a function pointer fits into one machine word, the *v-table* adds 5 hidden words of memory overall.

```
// FunctionPtr is a generic pointer to a generic member function
FunctionPtr Base::__vtable[5] = {
    &Base::virt0, &Base::virt1, &Base::virt2,
    &Base::virt3, &Base::virt4 };
```

Second step: adding an hidden pointer
An hidden pointer called *v-pointer* is added by the compiler to each object of `Base` class as it was an hidden data member.

```
// Original C++ source code
class Base
{
    public:
    // ...
        FunctionPtr* __vptr;  // Supplied by the compiler,
                              // hidden from the programmer
    // ...
};
```

Third step: initializing `this->__vptr`

Compiler initializes `this->__vptr` within each constructor in order to let each object's `v-pointer` to point at its class `v-table`. This step could be thought of as if the following instruction is added in each constructor's *init list*.

```
Base::Base( /*...arbitrary params...*/ )
  : __vptr(&Base::__vtable[0])  // Supplied by the compiler,
                                // hidden from the programmer
  // ...
{
  // ...
}
```

When the code defines a `Der` class which inherits from the `Base` class, the compiler repeats the first and third steps, but *skips the second step*.

In the **first step**, compiler creates another hidden *v-table*, in which the compiler replaces the virtual methods overridden in the derived class. For example, in case `Der` overrides `virt0()...virt2()` the `Der`'s *v-table* looks like

```
// Pseudo-code (not C++, not C)
// for a static table defined within file Der.cpp
FunctionPtr Der::__vtable[5] = {
    &Der::virt0, &Der::virt1, &Der::virt2,
    &Base::virt3, &Base::virt4 };
```

In the **third step**, the compilers for each `Der` object changes the `v-pointer` in order to make it point to its class `v-table`.

At this point, it is possible to call a virtual function with code like

```
void mycode(Base* p)
{
    p->virt3();
}
```

The compiler rewrites the code like

```
void mycode(Base* p)
{
    p->__vptr[3](p);
}
```

At this point

- a *first load* gets the `v-pointer` and stores it in a register which we call `r1`
- a *second load* gets the work at `r1 + 3*4` (if function-pointers are 4-bytes long) and stores it in a second register `r2`
- *compiler calls* the code at location `r2`

Therefore, it is possible to state that:

- a *minimal space-overhead* is needed to manage `virtual` functions
- `virtual` function calls are almost *as fast as* non-`virtual` function calls

- there is *no chaining effect*, i.e. the fetch-fetch-call mechanism is not dependent on how deep the inheritance gets.

Calling a Base Class `virtual` member function

When a `virtual` function is called by its *fully-qualified name*, the compiler skip the `virtual` call mechanism and uses the same mechanism used for non-`virtua` functions. Therefore, the fully-qualified `virtual` function is called by name rather than by slot-number.

To call the `virtual` function defined in the `Base` within the derived class, it is possible to write

```
void Der::f()
{
    Base::f();
}
```

`virtual` destructor

A `virtual` destructor is required any time someone `deletes` a derived-class object through a base-class pointer. In this case, indeed, the destructor invoked matches the type of the pointed object ignoring the type of the pointer itself.

In general, a `virtual` destructor is needed

- if the class is meant to be *derived from*
- *and* if code uses `new Derived`
- *and* if code uses `delete p` in which `p` is a pointer to `Base` pointing to an actual object of `Derived` class.

To be shorter, destructor needs to be `virtual` any time there is a `virtual` method in the class. Indeed

- it's a general safety measure in case the class will be used as a base class;
- it costs nothing since the

`virtual` constructor

An idiom that allows to achieve a behavior that C++ does not really support.

virtual-ctors

copy-of-abc-via-clone

Inheritance - Proper Inheritance and Substitutability

ISOCpp FAQ about **Inheritance - Proper Inheritance and Substainability**

Converting `Derived**` → `Base**`

Differently from the conversion `Derived*` → `Base*`, the conversion mentioned in the title does not compile, because it would be dangerous.
Even though a `Derived` object is a kind of `Base` object, in case the conversion in the title was possible the `Base**` could be dereferenced and it could be possible to make `Base*` point to an object of a `different derived class`.
This prohibition has similarities with 1.3.

container of `Thing` == kind-of container of `Everything`?

As a consequence of 1.4, a container of a certain `class Thing` objects is not a container of `class Anything` objects, even if `Thing` is a kind of `Anything`.

Is an array of `Derived` a kind-of array of `Base`?

In the same perspective of the previous two paragraphs, the answer is negative and easy to explain: a `Derived` object is larger than a `Base` object and it would mess up the pointer aritmetics in case it was possible to store both the kinds of objects in the same array.

Inheritance - Abstract Base Classes (ABCs)

ISOCpp FAQ about **Inheritance - Abstract Base Classes**
At the design level, an `ABC` corresponds to an abstract concept. At the programming language level, an `ABC` is a class containing one or more pure `virtual` functions, as in the following example

```
class Shape
{
public:
    virtual void draw() const = 0;  // = 0 means "pure virtual"
    // ...
};
```

Even though it is possible to provide a definition of a pure `virtual` function, it could be confusing.

Copy Constructor and Assignment Operator for a class containing a pointer to a (abstract) base class

If a class owns 1.4

Inheritance - What your mother never told you

ISOCpp FAQ about **Inheritance - What your mother never told you**

`final` classes and `virtual` methods

Calling `virtual` function from non-`virtual` functions of base class

Inheritance - `private` and `protected` inheritance

ISOCpp FAQ about **Inheritance - `private` and `protected` inheritance**

Private and protected inheritance are expressed using respectively the keyword `private` and `protected` in place of `public` in the definition of the derived class.
`private` inheritance is a syntactic variant of the concepts of composition or aggregation. For example, the "`Car` **has-a** `Engine`" relationship can be implemented both using single composition and `private` inheritance. For the latter, an example is

```
class Car : private Engine
{
    public:
    Car() : Engine(8) {}  // in case Engine has a ctor
                          // which takes an int
    using Engine::start;
}
```

`private` inheritance has both similarities and differences when compared to composition. Some of the similarities are

- a single `Engine` is present in both cases;
- in neither case a conversion `Car* → Engine*` can be done by users;
- `Car` has a `start()` method that calls `Engine`'s `start` method.

The differences are

- simple-composition is needed in case multiple `Engine` are contained in a `Car`;
- `private` inheritance can lead to unnecessary highlightbfmultiple inheritance
- `private` inheritance allows member of `Car` to convert `Car* → Engine*`
- `private` inheritance allows access to `protected` members of the base class
- `private` inheritance allows `Car` to override `Engine`'s `virtual` functions
- `private` inheritance make it simpler to implement a `Car`'s `start()` method that calls the `Engine`'s `start()` method.

Which should I prefer: composition or private inheritance?

Composition should be generally preferred to `private` inheritance. The latter gives access to the internals and there could be many class to expose. This aspect could make the program harder to maintain and easy to break when something in the code changes. A scenario in which `private` inheritance is *preferred* is when a `class Foo` uses code in `Class Bar` and the code in the latter class invokes member functions in the former class. The solution is to let `class Foo` call non-`virtual` in `class Bar`, and `class Bar` calls its own (pure) `virtual`s, which are overridden by `class Foo`.

Should I pointer-cast from a private derived class to its base class?
It is generally not a good practice.
The conversion `PrivatelyDer* → Base*` (and likewise for the references) is safe and no cast is needed or recommended. However, users of `PrivatlyDer` should avoid this conversion, since a private decision is subject to change without notice.

How is `protected` inheritance related to `private` inheritance?
Similarly, `protected` inheritance allows to override `virtual` functions in the base class and does not claim the derived is a kind-of-base class.
Dissimilarly, `protected` inheritance allows derived classes of derived classes to know about the inheritance relationship. The `benefit` is the capability of the *protected derived class* to exploit the relationship to the base class. The *cost* is the fact that changes the relationship in the protected derived class can potentially break further derived classes.

Access rules with `private` and `protected` inheritance
Considering the following code

```
class B                    { /*...*/ };
class D_priv : private   B { /*...*/ };
class D_prot : protected B { /*...*/ };
class D_publ : public    B { /*...*/ };
class UserClass            { B b; /*...*/ };
```

- none of the derived classes can access anything `private` in `B`;
- in `D_priv`, the `public` and `protected` of `B` parts are `private`
- in `D_prot` the `public` and `protected` parts of `B` are protected;
- in `D_publ` the `public` parts of `B` are public and the `protected` parts are protected (`D_publ` is-a-kind-of-a `B`);
- in `class UserClass` can accesso only the `public` parts of `B`.

The correct way to make a `public` member `f(int, float)` of `B` public in `D_prot` or in `D_priv` makes use of the **scope operator** as the in the following code

```
class D_prot : protected B
{
public:
    using B::f;
}
```

Inheritance - Multiple and Virtual Inheritance

ISOCpp FAQ about **Inheritance - Multiple and Virtual Inheritance**

Do we really need multiple inheritance?
Every modern language with `static` type checking and inheritance provides multiple inheritance. In C++, an *ABC* serves usually as an interface and class can need many. Other language simply have a separate name for pure abstract classes interfaces. Using multiple inheritance is not really common but it is better than using workarounds.

Disciplines for using multiple inheritance

- Inheritance should be used if it removes `if`/`switch` statements. Indeed, while composition is meant for code reuse, inheritance's primary purpose is dynamic binding and the flexibility which comes with it.
- *pure ABC*s play a crucial role in the context of `multiple inheritance`, since classes with as little data as possible and almost all pure `virtual` methods help avoiding inheritance for code reuse and using it for interface sustainability'
- The **bridge pattern** or **nested generalization** are possible alternatives and a wise designer checks out all the ways before choosing the best.

Example for the guidelines
Imagine a context in which there are multiple variants of a similar concepts, for example land, water, air and space vehicles, and different power sources, for example gas, wind, nuclear power and so on.
Before tieing up everything using multiple inheritance, there are two aspects to take into account choosing a good strategy:

- users need to call methods on a `Vehicle`-reference and expect the implementation of those methods to be specific to a concrete vehicle, like `LandVehicle`;
- users need a `Vehicle`-reference that refers to a concrete vehicle, for example `GasPoweredVehicle`, and want to call methods on that `Vehicle`-reference expecting the implementation to get overridden by `GasPoweredVehicle`.

In case both the conditions stand, multiple inheritance could be the correct choice. Better alternatives could still be available depending on a few more decision criteria.
In case there are **N** geographies (land, water, air, space, e.c.) and **M** power sources, there are at least 3 options.

- **Bridge Pattern**
  `Vehicle` and `Engine` are two pure `ABC`s, concrete vehicles and engines are derived from. `Vehicle` interface has an `Engine*` and everything is matched at runtime.
  This way the code grows *gracefully*, because there are only **N+M** derived classes to implement. There are several desadvantages as well. A first one is due to having at most **N+M** overrides and therefore the same number of concrete algorithms and

adding more could be very difficult.
Bridge Pattern is not able to solve nonsensical choices, as a pedal powered space vehicles and to solve this issue extra checks are needed as well.
Another issue is the absence of an interface for all the gas powered vehicles, which are considered just a kind of `Vehicle`. On the other side, all gas powered vehicles share their code in derived class `GasPoweredEngine`.

- **Nested Generalization**
  One of the hierarchies is choosen as primary, and suppose to choose the geography as primary in the current example.
  Concrete classes as `LandVehicle` and `WaterVehicle` are derived from `Vehicle` and each would have further derived classes, one per power source. For example, `LandVehicle` would be the base class for `GasPoweredLandVehicle`, `NuclearPoweredVehicle` and so on.
  This way, the codebase does not grow gracefully because **NxM** different derived classes need to be implemented, the advantage is the possibility to have the same number of different algorithms.
  This approach provides with a fine granular control removing the nonsensical combination because the combinations are decided in the design and should be reasonable.
  Analougusly to *Bridge Pattern*, there is not a common base class for the specific vehicles, and finally it is not obvious how to share code between different vehicles with same power source.

- **Multiple Inheritance**
  There are two different hierarchies as in the *Bridge Pattern*, the `Engine*` is no longer a data member of the vehicle classes though, and in its place roughly **NxM** derived classes are implemented below both the hierarchy of the geographies and the hierarchy of egines.
  In this context, the concept of `Engine` changes and vehicle classes should be renamed, e.g. `GasPoweredEngine` would be renamed to `GasPoweredVehicle`. Moreover, `GasPoweredLandVehicle` multiply inherits from `GasPoweredVehicle` and `LandVehicle`. At the cost of not having a graceful growth, the gain is a fine granular control simply because nonsesincal combinations like `PedalPoweredSpaceVehicle` are not created for the user.
  In this model, it is also possible for any gas powered vehicle to share a common interface. Unlike the *Bridge Pattern*, the derived class can override the shared code to replace parts that are not ideal for the new class.

Visualization of tradeoffs

Given **N** and **M**, the tradoffs of the approaches described in 1.4 are summarized in table 9.1

The meaning of the colums are the following

	Grow Gracefully	Low Code Bulk	Fine-Grained Control	Static Detect Bad Combos	Polimorphic on both sides	Share Code
Bridge						
Nested Gen.						
Multiple In.						

Table 9.1: *Bridge Pattern, Nested Generalization and Multiple Inheritance's tradeoffs.*

- **Grow Gracefully**
  For the example of the previous paragraph, the growth is due to adding more geographies or power sources and in this case it is linear.
- **Low Code Bulk**
- **Fine Grained Control**
  It takes into account the option of having a different algorithm for any of the different **NxM** possibilities or being stuck with a single solution for all the concrete classes of vehicles.
- **Static Detect Bad Combos**
- **Polymorphic on Both Sides**
- **Share Common Code**

Second Example for the guidlines
This example is more symmetric than the previous one, resulting in being better managed in using a multiple-inheritance solution.
Suppose to start with two categories of vehicles, for example `LandVehicle` and `WaterVehicle`, and later an `AmphibiousVehicle` is needed as well.
Given this coditions

- it is really needed to have an amphibious vehicle, instead of using the other classes with a bit of changes;
- users need a `LandVehicle` reference that refers to an `AmphibiousVehicle` and need to call methods on the reference expecting the actual implementation is specific to the referenced object;
- same as the previous point, with `WaterVehicle`.

in this case the *multiple inheritance* would be the best choice. Answering the question seen in 1.4 could help being more confident with this choice.

The Dreaded Diamond

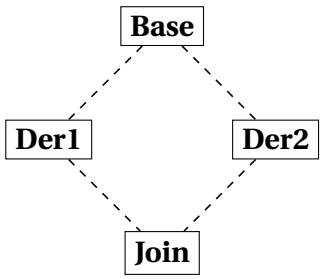


Figure 9.1: *A Dreaded Diamond.*

A *dreaded diamond* describes a class structure in which a particular class appears more than once in an inheritance hierarchy and looks like the diagram in figure 9.1.
Actually, C++ provides techniques to deal with the dreads of this triangle and this situation is not really dreaded but just something to be aware of.
The issue is due to `Base` being inherited twice with the consequence that any data member declared in `Base` appears twice within a `Join` object.
This duality creates ambiguities: when `Join` change a `Base`'s data member, it is not clear which derived class to go through for the change; likewise, a conversion `Join* → Base*` is ambigous.
C++ lets resolve the ambiguity with the *scope operator*, there is a better solution though.

`virtual` Inheritance

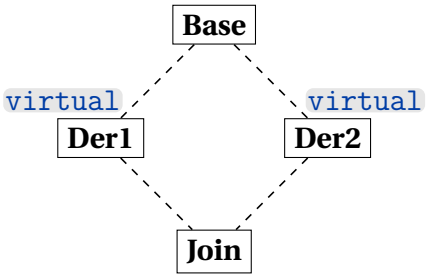


Figure 9.2: *A Nicer Diamond.*

To avoid duplicates in the `Join class`, every class that derives from `Base` needs to `virtual`-inherit. This way, an instance of `Join` will contain only one `Base` subobject and this solution is better than using fully qualified names.

delegate to a sister class

`virtual` inheritance helps achieving delegation to a sister class.

```
class Base
{
public:
    virtual void foo() = 0;
    virtual void bar() = 0;
};

class Der1 : public virtual Base
{
public:
    virtual void foo();
};
void Der1::foo()
{ bar(); }

class Der2 : public virtual Base
{
public:
    virtual void bar();
};

class Join : public Der1, public Der2
{
public:
    // ...
};
```

`Join` objects contain the `Der1`'s overridden `foo()` and the `Der2`'s overridden `bar()`. For this reason, when `Der1::foo()` calls `this->bar()` it ends up calling `Der2::bar()`.

Considerations needed when

- **using `virtual` inheritance**

Usually, `virtual` base classes and the ones that directly inherits from them are pure abstract classes, and contain very little if any data.

- **inheriting from a class that uses `virtual` inheritance**

The initialization list of the most-derived-class's constructor directly invokes the virtual base class's constructor.
C++ has a set of rules to make sure the `virtual` base class's constructor and destructor get called once per instance

- `virtual` base classes are constructed befor all non-highlightttvirtual base classes.
- constructors for `virtual` base classes anywhere in you class's inheritance hierarhy are called by the most derived class's constructor.

A consequence is the need to pass whatever parameters are required to call the `virtual` base class's constructor. In case of multiple `virtual` base classes, the most-derived class's constructor needs more parameters than other contexts require.
The practice of leaving the `virtual` base class without any data to initialize implies that the constructor probably takes no parameters.

- **using a class that uses `virtual` inheritance**

`dynamic_cast` is preferable to C-style downcasts.

Order of constructors in a multiple and/or `virtual` inheritance

First constructors to be executed are the virtual base classes' ones, in the order of appearance of base class names in the declaration of the derived class.
After the previous step, the construction order is from base class to derived class. Indeed, the compiler at first makes an hidden call to the non-virtual base classes' constructor in the derived class's constructor.
The previous rule is applied recursively: if `B1` inherits from `B1a` and `B1b`, and `B2` inherits from `B2a` and `B2b`, the final order of initialization is:
`B1a → B1b → B1 → B2a → B2b → B2 → D`.
The last order is determined by the order that the base classes appear in the declaration of the class.

Order of destructors in a multiple and/or `virtual` inheritance

For what concerns the non-virtually inheritance, the destructor order is the opposite of the constructor order:
`D → B2 → B2b → B2a → B1 → B1b → B1a`
Afterwards, the destructor of the virtual base classes that appear in the hierarchy are called, taking into account a class only when it uniquely appear for the first time.

2 C++11 New Features

2.1 Value Categories

Cppreference documentation about **value categories**.
Due to the expressiveness of the C++ language, the terms **lvalue** and **rvalue** references evolved in a more complex set.

glvalues

Generalized lvalue is used to refer to something that could be either a **lvalue** or **xvalue**.

rvalues

This term is inherited from C, in which **rvalues** can be on the right side of an assignment. In C++ this term can refer to things that are either **xvalues** and **prvalues**.

lvalues

This term is also inherited from C, in which **lvalues** can be on the left side of an assignment. Therefore, the simplest one is the name of a variable in a declaration.
Any expression resulting in an *lvalue reference* is also an *lvalue*.

```
int v;          // v is an lvalue of type int
A*  p1;         // *p1 is an lvalue
A&  r1 = v1;    // r1 is an lvalue
A&  f();        // f() is an lvalue
```

Accessing memebers of objects through pointers or references yields *lvalues*. For example, with the following code

```
struct Foo
{
    Bar a;
    Bar b;
};

Foo& f2();
Foo* p2;
Foo  v2;
```

`f2().a`, `p2->b` and `v2.a` are all *lvalues* if type A.
String literals are *lvalues* of type array.
To conclude, a **named object** declared with an *rvalue reference* is an *lvalue*. The name rvalue reference indicates that it can bind to an *rvalue*, and once you've declared a variable and given it a name it is an *lvalue*.

```
void Foo(A&& bar)
{
    // within Foo, bar is an lvalue
    // which will only bind to rvalues
}
```

xvalues

This term indicates eXpiring value: an unnamed object that will be destroyed soon. **xvalues** can be treated either as *glvalues* or *rvalues*, based on the context.
Usually, *xvalues* only arise through *explicit casts* and *function calls*. If an expression is cast to an *rvalue* reference to some type `T` then the result is an *xvalue* of type `T`

```
static_cast<A&&>(v1) // yields an xvalue of type A
```

If the return type of a function is an *rvalue reference* to some type **T**, the result is an *xvalue* of type **T**. For example, the `std::move` is declared as

```
template <typename T>
constexpr remove_reference_t<T>&& move(T&& t) noexcept;
```

`std::move(v1)` is an **xvalue** of type `A`. The type deduction rules deduce **T** to be `A&` because `v1` is an **lvalue**.

prvalues

Pure *rvalue* is an *rvalue* which is not an *xvalue*
Literals other than string literals are *pvalues*. For example, 42 is a *prvalue* of type `int` and $3.141f$ is a *prvalue* of type `float`
Temporaries are *prvalues*. For example any temporaries created as a result of an implicit conversion is also a *prvalue*. In the following code

```
int consume_string(std::string&& s);
int i = consume_string("hello");
```

the string literal in the function call will implicitly convert to a temporary of type `std::string`, which can bind to the rvalue reference used for the function parameter since the temporary is a *prvalue*

2.2 Reference Binding

cppreference documentation about reference declaration.
The *main difference* between *lavalues* and *rvalues* is the way they bind to references. The differences in type deduction rules can have a big impact too.

2.3 lvalue references

These references are declared with a single ampersand &.
A **non-const lvalue reference will only bind to non-const lvalues** of the same type, or a class derived from the referenced type

```
class C : public A{};

int i = 42;
A a;
B b;
C c;
const A ca{};
```

```
8
9  A&    r1  = a;
10 A&    r2  = c;
11 A&    r3  = b;      // error, wrong type
12 int&  r4  = i;
13 int&  r5  = 42;     // error, cannot bind rvalue
14 A&    r6  = ca;     // error, cannot bind const object
15                     // to non const ref
16 A&    r7  = r1;
17 A&    r8  = A();    // error, cannot bind rvalue
18 A&    r9  = B().a; // error, cannot bind rvalue
19 A&    r10 = C();    // error, cannot bind rvalue
```

A **const lvalue reference can also bind to rvalues**. The object bound to the reference must have the same type as the referenced type or a class derived from the referenced type. It is possible to bind both const and non-const values to a const lvalue reference.

```
1  const A&    cr1  = a;
2  const A&    cr2  = c;
3  const A&    cr3  = b;      // error, wrong type
4  const int&  cr4  = i;
5  const int&  cr5  = 42;     // rvalue can bind OK
6  const A&    cr6  = ca;     // OK, can bind const object to const ref
7  const A&    cr7  = cr1;
8  const A&    cr8  = A();    // OK, can bind rvalue
9  const A&    cr9  = B().a; // OK, can bind rvalue
10 const A&    cr10 = C();    // OK, can bind rvalue
11 const A&    cr11 = r1;
```

Binding a temporary object (i.e. a *prvalue*) to a const lvalue reference *extends the lifetime* of that temporary to the lifetime of the reference. In the previous example, it is possible to use `r8`, `r9` and `r10` later in the code.
Lifetime extension does not extend to references initialized from the first reference. If a function parameter is a const lvalue reference and gets bound to a temporary passed to the function call, the temporary is destroyed when the function returns. This happens also if the reference was stored in a longer-lived variable, such as a member of a newly constructed object.
`volatile` and `const volatile` lvalue references are much less used but behave as expected: volatile `T&` bind to a volatile or non-volatile, non-const lvalue of type T or a class derived from T. However, volatile const lvalue references do not bind to rvalues.

2.4 rvalue references

An *rvalue reference* can be used to extend the lifetime of temporary objects, making the referenced object modifiable through them. The reference must be const in order to bind to a const object, though const rvalue reference are much rarer.

```
1  const A make_const_A();
2  A&&       rr1  = a;                // error, cannot bind lvalue
3                                     // to rvalue reference
```

```
A&&       rr2  = A();
A&&       rr3  = B();              // error, wrong type
int&&     rr4  = i;                // error, cannot bind lvalue
int&&     rr5  = 42;
A&&       rr6  = make_const_A();   // error, cannot bind
                                   //        const object
                                   //        to non const ref
const A&& rr7  = A();
const A&& rr8  = make_const_A();
A&&       rr9  = B().a;
A&&       rr10 = C();
A&&       rr11 = std::move(a);     // std::move returns an rvalue
A&&       rr12 = rr11;             // error rvalue references
                                   //       are lvalues
```

rvalue references extend the lifetime of temporary objects in the same way const lvalue references do, therefore temporaries associated with `rr2`, `rr5`, `rr7`, `rr8`, `rr9` and `rr10` remain valid until the corresponding references are destroyed.

2.5 Reference Collapsing

In C++ it is possible to form reference to reference in *templates* and `typedef` expressions. In this context, the refefrence collapsing rule apply: rvalue reference to rvalue reference collapses to an rvalue reference, all the others collapse to lvaue references.

```
typedef int&  lref;
typedef int&& rref;
int n;

lref&  r1 = n; // type of r1 is int&
lref&& r2 = n; // type of r2 is int&
rref&  r3 = n; // type of r3 is int&
rref&& r4 = 1; // type of r4 is int&&
```

Implicit conversion

A reference-to-A can be initialized with any D object which has an inplicit conversion operator that returns `A&`. In this case the reference is bound to the result of the conversion operator.

A const lvalue-reference-to-E will bind only to objects implicitly convertible to E. The reference is bound to the temporary resulting from the conversion.

General properties

In general, *rvalues* can't be modified and their address can't be taken.

However, as long as class `X` has a member function named `do_something`, the expression `X().do_something` is valid.

ref qualifiers along with `const` can be used to tag different overloads of class member functions, to indicate they can be applied to only *lvalues* or to only *rvaules*.

When the type of a variable or function template parameter is deduce from its initializer, whether the initializer is an lvalue or rvalue can affect the deduced type.

2.6 Move Semantic

Move semantics is a new way of moving resources around in an optimal way, since it helps *avoiding unnecessary copies* of temporary objects. It is based on rvalue references and plays an *important role in exception safety* for C++.

2.7 Perfect Forwarding

Cppreference documentation about **forward** utility.
Cppreference about **forwarding references**.
Perfect forwarding is an important C++0x feature built on top of rvalue references. It provides the programmer with the capability automatically apply the move semantics, even when there are intervening function calls to separate the source and the destination of a move.
Examples include *constructors* and *setter functions*, that can forward arguments they receive directly to the data member they are initializing or setting. Standard Library functions like `make_shared` "perfect-forwards" its arguments to the class constructor of whatever object the to-be-created `shared_ptr` is to point to.
The idiomatic expression of the perfect forwarding make use of the `std::forward`

```
template <typename T>
void wrapper(T&& arg)
{
    wrapped(std::forward<T1>(t1), std::forward<T2>(t2));
}
```

2.8 `auto`

Cppreference documentation about `auto`.
References in bibliography: [49], [50], [51].
This placeholder type specifier has the following meanings:

- for **variables**:
  the variable that is being declared will be automatically deduced from its initializer;
- for **functions**:
  the return type will be deduced from its return statements;
- for **non-type template parameters**:
  the type will be deduced from the argument.

Following a summary of contextes in which `auto` may appear.
In the summary, the rules for template argument deduction are referenced.

- if used **in the type specifier of a variable** the variable type is deduced from the initializer list using the template argument deduction:

```
auto x = expr;

```

In case *no pointer or reference* is attached to `auto`, both `const` and references are ignored:

```
int y = 10;
int& r = y;
auto x = r;                 // type is int (reference ignored)

const int y = 10;
auto x = y;                 // type is int (const ignored)

int y = 10;
const int& r = y;
auto x = r;                 // type is int
                            // (const and reference ignored)

const int a[10] = {};
auto x = a;                 // type is int*
                            // (array to pointer conversion)

```

Modifiers which may accompany `auto` will partecipate in the type deduction. For example, in the code

```
const auto& i = expr;
template<class U> void f(constU& u);

```

the type deduced for the first code line is the same deduced for the argument `u` of the second line if the function call `f(expr)` was compiled.
Therefore `auto&&` may be deduced both as an *lvalue* reference and an *rvalue* reference.
In case `auto` is used to declare multiple variables, the deduced type must match.

- If used **in a function defined with the trailing return type syntax**, it is just part of the syntax and automatic type deduction is not performed

```
auto function_name(parameter_list) -> return_type
{
    // Function implementation
}

```

- If used **as return type of a function defined without the trailing return type**, or **as a return type of a lambda expression**, the keyword `auto` indicates that the return type will be deduced from the return statement, using the rules for template argument deduction.

- If used **in combination with `decltype`** and the declared type of the variable is `decltype(auto)`, the keyword `auto` is replaced with the expression (or expression list) of its initializer and the actual type is deduced using the rules for `decltype`.

- If used **along with `decltype` in the return type of a function** and the return type of the function is `decltype(auto)`, the keyword `auto` is replaced with the operand of its return statement, and the actual return type is deduced using the rules of `decltype`.

- If used in the **type-id of a `new` expression**, the type is deduced from the initializer.

- If used **in the parameter declaration of a non-type template parameter.**, for example `template<auto I> struct A;`, its type is deduced from the corresponding argument.

- if used **as a nested scope like in `auto::`** ???

- if used **in the parameter declaration of a lambda-expression**, such lambda expression is a *generic lambda*. This means that the folowing generig lambda will be written by the compiler as an instance of the templates which follows

```
auto g_lambda = [] (auto a) { return a; } ;
class /* unnamed */
{
public:
    template<typename T>
    T operator () (T a) const { return a; }
};

```

- if used **in a function parameter declaration**, like `void f(auto);`, the function declaration introduces an abbrevited function template [61]

2.9 `decltype`

`decltype` inspects the declared type of an entity or the type and value category of an expression.

Explain difference between parenthesized

See [62]

2.10 Initializer list

Cppreference documentation about Initializer list

An object of type `std::initializer_list<T>` is a lightweight proxy object that provides access to an array of objects of type `const T`.

2.11 Uniform initalization syntax and semantics

C++98 provides the users with several ways of initializing an object. These ways depend on the type and the initialization context and have different ways of working.
Some examples can be the following code lines

```
string a[] = { "foo", " bar" };         // ok: initialize array
vector<string> v = { "foo", " bar" };   // error: initializer list
                                        // for non-aggregate vector
void f(string a[]);
f( { "foo", " bar" } );                 // syntax error:
                                        // block as argument
```

and

```
int a = 2;              // "assignment style"
int aa[] = { 2, 3 };    // assignment style with list
complex z(1,2);         // "functional style" initialization
x = Ptr(y);             // "functional style" for
                        // conversion/cast/construction

int a(1);               // variable definition
int b();                // function declaration
int b(foo);             // variable definition or function declaration
```

C++11 introduces a {}-initializer, which provides a much simpler and consistent way to perform initialization, for all different initialization context as in the following example

```
X x1 = X{1,2};
X x2 = {1,2};    // the = is optional
X x3{1,2};
X* p = new X{1,2};
struct D : X
{
    D(int x, int y) :X{x,y} { /* ... */ };
};
struct S
{
    int a[3];
    S(int x, int y, int z) :a{x,y,z} { /* ... */ }; // solution to
                                                    // old problem
};
```

{}-initializer produces the same result in any context

```
X x{a};
X* p = new X{a};
z = X{a};           // use as cast
f({a});             // function argument (of type X)
return {a};         // function return value (function returning X)
```

{}-initializer is also useful to solve the most vexing parse problem, explained in 1.2.

2.12 Range `for`

A `range-for statement` applies to all standard containers and string as well, providing a less-typing way to `for`-loop over a *range* of elements.
An example of usage in the following code

2.13 `noexcept`

Cppreference about `noexcept`

2.14 `nullptr`

Cppreference documentation about **nullptr**
The keyword `nullptr` denotes the pointer literal. It is a `prvalue` of type `std::nullptr_t`. There exist implicit conversions from `nullptr` to null pointer value of any pointer type and any pointer to member type. Similar conversions exist for any null pointer constant, which includes values of type `std::nullptr_t` as well as the macro NULL.

2.15 constexpr

Cppreference about `constexpr`
The constexpr specifier declares that it is possible to evaluate the value of the function or variable at compile time. Such variables and functions can then be used where only compile time constant expressions are allowed (provided that appropriate function arguments are given).

2.16 Lambda expressions

Cppreference about **lambda expressions**

A lambda expression is a closure, i.e. an unnamed

captures	a comma-separated list of zero or more captures, optionally beginning with a capture-default A lambda expression can use a variable without capturing it if the variable • is a non-local variable or has static or thread local storage duration (in which case the variable cannot be captured), or • is a reference that has been initialized with a constant expression. A lambda expression can read the value of a variable without capturing it if the variable • has const non-volatile integral or enumeration type and has been initialized with a constant expression, or • is constexpr and has no mutable members.
tparams	a non-empty comma-separated list of template parameters used to provide names to the template parameters of a generic lambda
t-requires	adds constraints to tparams
front-attr	since C++23 **study**
params	the parameter list of `operator()` of the closure type
specs	A list of the following specifiers, each specifier is allowed at most once in each sequence. If not provided, the objects captured by copy are `const` • `mutable` allows body to modify the objects captured by copy • `constexpr` since C++17 • `consteval` since C++20 • `static` since C++23
exception	provides the dynamic exception specification or (until C++20) the noexcept specifier for operator() of the closure type
back-attr	an attribute specifier sequence applies to the type of operator() of the closure type (and thus the `[[noreturn]]` attribute cannot be used)
trailing-type	`-> ret` where `ret` specifies the return type
requires	adds constraints to `operator()` of the closure type
body	function body

3 C++20 New Features

https://www.geeksforgeeks.org/features-of-c-20/ https://en.cppreference.com/w/cpp/20

3.1 Concepts

3.2 Coroutines

3.3 Modules

CHAPTER 10

Concurrency

C++ includes built-in support for threads, atomic operations, mutual exclusion, condition variables, and futures.

1 Threads

thread	manages a separate thread
jthread	thread with support for auto-joining and cancellation

1.1 Functions managing the current thread

yield	suggests that the implementation reschedule execution of threads
get_id	returns the thread id of the current thread
sleep_for	stops the execution of the current thread for a specified time duration
sleep_until	stops the execution of the current thread until a specified time point

2 Atomicity

2.1 Atomic operations

These components are provided for fine-grained atomic operations allowing for lockless concurrent programming. Each atomic operation is indivisible with regards to any other atomic operation that involves the same object. Atomic objects are free of data races.

2.2 Atomic types

atomic	atomic class template and specializations for bool, integraln and pointer types
atomic_ref	provides atomic operations on non-atomic objects

2.3 Operations on atomic types

atomic_is_lock_free	checks if the atomic type's operations are lock-free
atomic_store atomic_store_explicit	atomically replaces the value of the atomic object with a non-atomic argument
atomic_load atomic_load_explicit	atomically obtains the value stored in an atomic object
atomic_exchange atomic_exchange_explicit	atomically replaces the value of the atomic object with non-atomic argument and returns the old value of the atomic
atomic_compare_exchange_weak atomic_compare_exchange_weak_explicit atomic_compare_exchange_strong atomic_compare_exchange_strong_explicit	atomically compares the value of the atomic object with non-atomic argument and performs atomic exchange if equal or atomic load if not
atomic_fetch_add atomic_fetch_add_explicit	adds a non-atomic value to an atomic object and obtains the previous value of the atomic
atomic_fetch_sub atomic_fetch_sub_explicit	subtracts a non-atomic value from an atomic object and obtains the previous value of the atomic
atomic_fetch_and atomic_fetch_and_explicit	replaces the atomic object with the result of bitwise AND with a non-atomic argument and obtains the previous value of the atomic
atomic_fetch_or atomic_fetch_or_explicit	replaces the atomic object with the result of bitwise OR with a a non-atomic argument and obtains the previous value of the atomic
atomic_fetch_xor atomic_fetch_xor_explicit	blocks the thread until notified and the atomic value changes
atomic_wait atomic_wait_explicit	replaces the atomic object with the result of bitwise XOR with a non-atomic argument and obtains the previous value of the atomic
atomic_notify_one	notifies a thread blocked in `atomic_wait`
atomic_notify_all	notifies all threads blocked in `atomic_wait`

2.4 Flag type and operations

atomic_flag	the lock-free boolean atomic type
atomic_flag_test_and_set atomic_flag_test_and_set_explicit	atomically sets the flag to true and returns its previous value
atomic_flag_clear atomic_flag_clear_explicit	atomically sets the value of the flag to false
atomic_flag_test atomic_flag_test_explicit	atomically returns the value of the flag
atomic_flag_wait atomic_flag_wait_explicit	blocks the thread until notified and the flag changes
atomic_flag_notify_one	notifies a thread blocked in atomic_flag_wait
atomic_flag_notify_all	notifies all threads blocked in atomic_flag_wait

2.5 Initialization

atomic_init	non-atomic initialization of a default-constructed atomic object
ATOMIC_VAR_INIT	constant initialization of an atomic variable of static storage duration
ATOMIC_FLAG_INIT	initializes an `std::atomic_flag` to false

2.6 Memory synchronization ordering

memory_order	defines memory ordering constraints for the given atomic operation
kill_dependency	removes the specified object from the `memory_order_consume` dependency tree
atomic_thread_fence	generic memory order-dependent fence synchronization primitive
atomic_signal_fence	fence between a thread and a signal handler executed in the same thread

2.7 Mutual exclusion

Mutual exclusion algorithms prevent multiple threads from simultaneously accessing shared resources. This prevents data races and provides support for synchronization between threads.

mutex	provides basic mutual exclusion facility
timed_mutex	provides mutual exclusion facility which implement locking with a timeout
recursive_mutex	provides mutual exclusion facility which can be locked recursively by the same thread
recursive_timed_mutex	provides mutual exclusion facility which can be locked recursively by the same thread and implements locking with a timeout
shared_mutex	provides shared mutual exclusion facility
shared_timed_mutex	provides shared mutual exclusion facility and implements locking with a timeout

2.8 Generic mutex management

lock_guard	implements a strictly scope-based mutex ownership wrapper
scoped_lock	deadlock-avoiding RAII wrapper for multiple mutexes
unique_lock	implements movable mutex ownership wrapper
shared_lock	implements movable shared mutex ownership wrapper
defer_lock_t try_to_lock_t adopt_lock_t	tag type used to specify locking strategy
defer_lock try_to_lock adopt_lock	tag constants used to specify locking strategy

2.9 Generic locking management

try_lock	attempts to obtain ownership of mutexes via repeated calls to `try_lock`
lock	locks specified mutexes, blocks if any are unavailable

2.10 Call once

once_flag	helper object to ensure that `call_once` invokes the function only once
call_once	invokes a function only once even if called from multiple threads

2.11 Condition variables

condition_variable	provides a condition variable associated with a `std::unique_lock`
condition_variable_any	provides a condition variable associated with any lock type
notify_all_at_thread_exit	schedules a call to `notify_all` to be invoked when this thread is completely finished
cv_status	lists the possible results of timed waits on condition variables

2.12 Futures

promise	stores a value for asynchronous retrieval
packaged_task	packages a function to store its return value for asynchronous retrieval
future	waits for a value that is set asynchronously
shared_future	waits for a value (possibly referenced by other futures) that is set asynchronously
async	runs a function asynchronously (potentially in a new thread) and returns a `std::future` that will hold the result
launch	specifies the launch policy for `std::async`
future_status	specifies the results of timed waits performed on `std::future` and `std::shared_future`

2.13 Future errors

future_error	reports an error related to futures or promises
future_category	identifies the future error category
future_errc	identifies the future error codes

2.14 Memory Order

Cppreference documentation about `std::memory_order`.
C++ defines several memory orders to specify how memory accesses are to be ordered around an atomic operation. Without any constraint, the apparent order of changes can even differ among multiple reader threads.
The defualt behavior provides for sequentially consistent ordering, This default behavior can reduce performance, therefore the possibility to give an additional `std::memory_order` argument to define the constraints that the compiler and processor must enforce beyond atomicity.

- `memory_order_relaxed`
  Relaxed operation: there are *no synchronization or ordering constraints* imposed on other reads or writes, only this operation's atomicity is guaranteed

- `memory_order_consume`
  A load operation with this memory order performs a consume operation on the affected memory location: no reads or writes in the current thread dependent on the value currently loaded can be reordered before this load. Writes to data-dependent variables in other threads that release the same atomic variable are visible in the current thread. On most platforms, this affects compiler optimizations only

- `memory_order_acquire`
  A load operation with this memory order performs the acquire operation on the affected memory location: no reads or writes in the current thread can be reordered before this load. All writes in other threads that release the same atomic variable are visible in the current thread

- `memory_order_release`
  A store operation with this memory order performs the release operation: no reads or writes in the current thread can be reordered after this store. All writes in the current thread are visible in other threads that acquire the same atomic variable and writes that carry a dependency into the atomic variable become visible in other threads that consume the same atomic

- `memory_order_acq_rel`
  A read-modify-write operation with this memory order is both an acquire operation and a release operation. No memory reads or writes in the current thread can be reordered before the load, nor after the store. All writes in other threads that release the same atomic variable are visible before the modification and the modification is visible in other threads that acquire the same atomic variable.

- `memory_order_seq_cst`
  A load operation with this memory order performs an acquire operation, a store performs a release operation, and read-modify-write performs both an acquire

operation and a release operation, plus a single total order exists in which all threads observe all modifications in the same order.

CHAPTER 11

Python

1 Decorator

A function decorator is a callable object that takes a function as its input parameter

decorator

Decorators with parameters

PythonDecoratorLibrary

2 Special Instance Method

Special Instance Method

3 Providing Multiple Ctors

multiple `__init__`

12

CHAPTER

Software Testing

13

CHAPTER

Embedded System

14

CHAPTER

AI

1) ML fundamentals
2) Pytorch, la libreria per ml in python
3) Ricetta per allenare modelli
4) Come il primo link, più in depth con più matematica
5) IL libro di introduzione
6) Implementazioni e spiegazioni da uno dei migliori ricercatori del campo

0.1 A

ADL • Argument Dependent Lookup
ADT • Abstract Data Type
Allocate • to assign, allot, distribute, or set apart for a particular purpose
Atomic • Objects of atomic types are the only C++ objects that are free from data races; that is, if one thread writes to an atomic object while another thread reads from it, the behavior is well-defined. In addition, accesses to atomic objects may establish inter-thread synchronization and order non-atomic memory accesses as specified by `std::memory_order`.

0.2 B

branch • a mean for teams to develop features giving them a separate workspace for their code. The branch to create are part of the branching strategy, which could be different depending on the specific svn used.

0.3 C

constexpr • a C++ specifier which declares that it is possible to evaluate the value of the function or variable at compile time.

0.4 D

data race •

deadlock • any situation in which no member of some group of entities can proceed because each waits for another member, including itself, to take action, such as sending a message or, more commonly, releasing a lock

Dependency injection • wikipedia

duck typing • a concept related to *dynamic typing*, where the type or the class of an object is less important than the methods it defines

double dispatch • wikipedia

dynamic binding •

0.5 E

encapsulation • preventing unauthorized access to some piece of information or functionality

0.6 H

Heap • a large pool of memory used to satisfy memory request by allocating part.

0.7 I

Idiom • a group of code fragments sharing an equivalent semantic role (meaning), which recurs frequently across software projects often expressing a special feature in one or more programming languages or libraries.

0.8 M

0.9 O

• region of storage with associated semantics

Overloading • a compile time polymorphism achieved specifying more than one function with the same name but different signature in the same scope.

Overridding • the act of redefining a method of a base class in a derived class, using the same name, return type and parameters.

0.10 P

Polymorphysm • ability of different objects to be accessed by a common interface.

0.11 R

Race Condition • undesirable situation which occurs when two instructions from different threads access the same memory location, at least one of these accesses is a write and there is no synchronization that is mandating any particular order among these accesses.

RAII • acronym for Resource Acquisition Is Initialization.

RBV optimization • see [87].

RVO • acronym for Return Value Optimization.

0.12 S

static typing •

SFINAE • acronym for **Substitution Failure Is Not An Error**, a technique used in template programming which discards the specialization from the overloaded set avoiding a compile error, when substituting the explicitly specified or deduced type for the template parameter fails.

Stack • LIFO-ordered contiguous region of temporary memory whose size is known to the compiler, which manages to allocate and deallocate blocks of memory for the stack variables like functions' local data. . If a region of memory lies on the thread's stack, that memory is said to have been allocated on the stack, i.e. stack-based memory allocation (SBMA).

Stub •

0.13 T

Test-driven Development • wikipedia

0.14 V

virtual member function • a mean to allow derived classes to replace the implementation provided by the base class. The compiler makes sure to call the correct replacement when the object is of the derived class and also when the object is accessed through a pointer to the base rather than to the derived class.

Bibliography

[1] Robert Sedgewick, Kevin Wayne - *Algorithms - 4th Edition*

[2] Robert C. Martin (2014) - *Principles of OOD*
Retrieved from: The OLD Object Mentor blog site

[3] Robert C. Martin (2009) - *Getting a SOLID start*
Retrieved from: Uncle Bob Consulting LLC.

[4] Sandu Metz (2009) - *SOLID Object-Oriented*
Retrieved from: Ghost Archive - Gotham Ruby Conference

[5] Robert C. Martin (2000) - *Design Principles and Design Patterns*
Retrieved from: Internet Archive - Objectmentor

[6] Robert C. Martin (2015) - *Single Responsibility Principle*
Retrieved from: Internet Archive - Objectmentor

[7] Robert C. Martin (2003) - *Agile Software Development, Principles, Patterns, and Practices*
Retrieved from: Internet Archive - Objectmentor

[8] *Open/Closed Principle*
Retrieved from: Internet Archive - Objectmentor

[9] *Liskov Substitution Principle*
Retrieved from: Internet Archive - Objectmentor

[10] *Interface Segregation Principle*
Retrieved from: Internet Archive - Objectmentor

[11] *Dependency Inversion Principle*
Retrieved from: Internet Archive - Objectmentor

[12] *Clean Architecture: A Craftman's Guide to Software Structure and Design*
Retrieved from: GunterMueller user GitHub

[13] *Clean Architecture: A Craftman's Guide to Software Structure and Design*
Retrieved from:

[14] *Pimpl Idiom*
Retrieved from: Wiki Wiki Web

[15] Herb Sutter - *The Fast Pimpl Idiom*
Retrieved from: Guru of the Week - 28

[16] Herb Sutter - *Compilation Firewalls*
Retrieved from: Guru of the Week - 24

[17] Herb Sutter - *Compilation Firewalls (C++11-updated version of GotW #24)*
Retrieved from: Guru of the Week - 100

[18] Herb Sutter - *The Joy of Pimpls (or more about the Compiler-Firewall Idiom).*
Retrieved from: More About Compiler Firewalls.

[19] *What is the copy and swap idiom?*
Retreived from: Stackoverflow.

[20] Herb Sutter - *Exception-Safe Class Design, Part I: Copy Assignment*
Retrieved from: Guru of the Week - 59

[21] Herb Sutter - *Exception-Safe Class Design, Part II: Inheritance*
Retrieved from: Guru of the Week - 60

[22] *What is the "Named Constructor Idiom"?*
Retrieved from: ISOC++ Wiki

[23] *Construct On First Use Idiom*
Retrieved from: ISOC++ Wiki

[24] *Nifty Counter Idiom*
Retrieved from: ISOC++ Wiki

[25] *Nifty Counter Idiom*
Retrieved from: Wikibook

[26] *Named Parameter Idiom*
Retrieved from: ISOC++ Wiki

[27] *Virtual Friend Function Idiom*
Retrieved from: ISOC++ Wiki

[28] *C++ Core Guidelines*
Retreived from: Standard Cpp Foundation Github.

[29] *Memory Management: Stack and Heap*
Retrieved from: CppCon YouTube Video

[30] *Leak-freedom in C++ . . . By Default - CppCon 2016*
Retrieved from: CppCon YouTube Video

[31] Boccara - *The Most Vexing Parse: How to Spot It and Fix It Quickly*
Retrieved from: Fluent C++

[32] *Most Vexing Parse*
Retrieved from:

C++ chapter bibliography

[33] R. Martinho Fernandes (2012) - *Rule of Zero*
Retrieved from: ISOC++ Blog

[34] Scott Meyers (2014) - *A Concern about the Rule of Zero*
Retrieved from: The View from Aristeia - Scoot Meyers' Activities and Interests

[35] *A polymorphic class should suppress public copy/move*
Retreived from: Standard Cpp Foundation Github

[36] *If you define or =delete any copy, move, or destructor function, define or =delete them all*
Retreived from: Standard Cpp Foundation Github

[37] *References*
Retrieved from: ISOC++ Wiki

[38] *Const Correctness*
Retrieved from: ISOC++ Wiki

[39] *Most vexing parse*
Retrieved from: WikiPedia

[40] (2016) - *Core C++ - lvalues and rvalues*
Retrieved from: Just Software Solutions

[41] (2008) - *Rvalue References and Perfect Forwarding*
Retrieved from: Just Software Solutions

[42] Peter Dimov, Howard E. Hinnant, Dave Abrahams - *The Forwarding Problem: Arguments*
Retreived from: open-std.org - N1385=02-0043

[43] Meyers, Sutter, Alexandrescu (2011) - *Session Announcement: Adventures in Perfect Forwarding*
Retrieved from: C++ and Beyond

[44] *What are the main purposes of std::forward and which problems does it solve?*
Retreived from: StackOverflow Thread

[45] *What is the move semantic?*
Retreived from: StackOverflow Thread

[46] Howard Hinnant (2014) - *Everything you wanted to know about Move Semantics*
Retreived from: Slideshare

[47] *If your function must not throw declare it noexcept*
Retreived from: Cpp Core Guidelines

[48] *What is the meaning of* `auto` *keyword*
Retrieved from: StackOverflow Thread

[49] Herb Sutter - `auto` *variables, Part 1*
Retrieved from: Guru of the weel - 92

[50] Herb Sutter - `auto` *variables, Part 2*
Retrieved from: Guru of the week - 93

[51] Herb Sutter - *AAA style (Almost Always* `auto`*)*
Retrieved from: Guru of the week - 94

[52] - *What's In a Class? - The Interface Principle*
Retrieved from: Guru of the week - C++ Report, 10(3), March 1998

[53] (2016)*Declaring non-type template arguments with* `auto`
Retrieved from: Programming Language C++, Evolution Working Group - P0127R1

[54] (2016)*Declaring non-type template parameters with* `auto`
Retrieved from: Programming Language C++, Evolution Working Group - P0127R2

[55] *Pros and Cons of Alternative Function Syntax in C++*
Retrieved from: Petr Zemek's Blog of a Software Engineer

[56] *ADL - Argument-dependent lookup*
Retrieved from: Cppreference

[57] *Lambda Expressions*
Retrieved from: Cppreference

[58] Larvi, Freeman, Crowl (2008) - *Lambda Expressions and Closures: Wording for Monomorphic Lambdas (Revision 4)*
Retrieved from: open-std.org

[59] Vandevoorde (2009) - *New wording for C++0x Lambdas*
Retrieved from: open-std.org

[60] *Closure (Computer Programming)*
Retrieved from: WikiPedia

[61] *Abbreviated function template*
Retrieved from: Cppreference

[62] `decltype` *specifier*
Retrieved from: Cppreference

[63] Stroustrup, Dos Reis (2007) - *Initializer list (Rev. 3)*
Retrieved from: open-std.org

[64] Merrill, Vandevoorde (2008) - *Initializer Lists - Alternative Mechanism and Rationale (v.2)*
Retrieved from: Initializer Lists — Alternative Mechanism and Rationale (v. 2

[65] Thorsten Ottosen (2007) - *Wording for range-based for-loop (revision 2)*
Retrieved from: open-std.org

[66] *Range-based `for` statements and ADL*
Retrieved from: open-std.org

[67] Svoboda (2010) - *Delete operators default to noexcept*
Retrieved from: open-std.org

[68] Mauer (2010) - *Deducing "noexcept" for destructors*
Retrieved from: open-std.org

[69] Abrahams, Sharoni, Gregor (2010) - *Allowing Move Constructors to Throw (Rev. 1)*
Retrieved from: open-std.org

[70] *Bjarne Stroustrup's C++ Style and Technique FAQ*
Retrieved from: stroutrup.com

[71] *Object-Oriented Programming*
Retreived from: Wikipedia - Object Oriented Programming

[72] *Object-Oriented Programming Using C++*
Retreived from: Weber State University - School of Computing - Computer Science

[73] *Allen Holub's UML Quick Reference*
Retreived from: Allen Holub website

[74] *Class Diagram*
Retreived from: WikiPedia - Class Diagram

[75] *State Diagram*
Retreived from: WikiPedia - State Diagram

[76] *State Machine*
Retreived from: WikiPedia - State Machine

[77] *Programming paradigm*
Retreived from: WikiPedia - State Machine

[78] *Composition over Inheritance*
Retreived from: WikiPedia - Composition over Inheritance

[79] *Delegation (object-oriented programming)*
Retreived from: WikiPedia - Delegation

[80] *Role-oriented programming*
Retreived from: WikiPedia - Programming paradigm

[81] James Rumbaugh (1991) - *Object-Oriented Modeling and Design*
Retreived from: Internet Archive

[82] McKenney, Bohem, Crowl (2008) - *C++ Data-Dependency Ordering: Atomics and Memory Model*
Retrieved from: ISO/IEC JTC1 SC22 WG21 N2556

[83] *Memory model synchronization modes*
Retrieved from: GNU gcc Wiki

[84] *What do each memory_order mean?*
Retrieved from: StackOverflow Thread

[85] *Python Glossary*
Retrieved from: docs.python.org

[86] *Open Lecture by James Bach on Software Testing*
Retrieved from: Eesti Infotehnoloogia Kolledž YouTube Channel

[87] *Does return-by-value mean extra copies and extra overhead?*
Retrieved from: ISOC++ Wiki

[88] - *SFINAE - Substitution Failure Is Not An Error*
Retrieved from: Cppreference